Together for Tomorrow!
Enabling People
Education for Future Generations

Khóa học AI

Đồ án tốt nghiệp
Báo cáo cuối khóa

Dành cho học viên (giảng viên đánh giá)

©2025 SAMSUNG. All rights reserved.
Samsung Electronics Corporate Citizenship Office holds the copyright of this document.
This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

VietTravel AI: Trợ lý du lịch có căn cứ dữ liệu và lập lịch trình tại Việt Nam

21/10/2026

Khóa học AI
Samsung Innovation Campus

Lớp:
SIC-AI-N02

Giảng viên:
Dr. Trần Quang Quý

Nhóm Travel AI

Thành viên:

Nguyễn Minh Phương
Thân Phú Cường

Tóm tắt

VietTravel AI là trợ lý du lịch xây dựng bằng Flask, hỗ trợ tìm kiếm địa điểm tại Việt Nam và tạo bản nháp lịch trình trong một ngày. Kho mã nguồn kết hợp hai luồng riêng: hỏi đáp du lịch bằng ngôn ngữ tự nhiên và lập lịch trình. Luồng hỏi đáp xác định ngôn ngữ, trích xuất đặc trưng liên quan đến chuyến đi, truy vấn dữ liệu PostgreSQL được nhập từ OpenStreetMap (OSM), rồi yêu cầu nhà cung cấp mô hình ngôn ngữ tạo câu trả lời nếu đã cấu hình thông tin xác thực. Luồng lập lịch truy vấn các điểm quan tâm (POI), mặc định tạo lịch bằng thuật toán xác định, kiểm tra bản nháp theo quy tắc về danh tính, thời gian và tọa độ, sau đó trả kết quả mà chưa lưu. Một thao tác xác nhận riêng, có xác thực, kiểm tra lại và lưu lịch trình vào lược đồ quan hệ của ứng dụng.

Báo cáo mô tả những gì có trong mã nguồn và các tệp kê khai dữ liệu đi kèm. Bản tổng hợp xử lý OSM trên toàn quốc ghi nhận 30.941 ứng viên đã lọc và 30.839 mã định danh duy nhất, nhưng các trường thống kê trùng lặp chưa khớp hoàn toàn với nhau. Kho mã nguồn có bộ dữ liệu thử nghiệm gồm 15 bản ghi và mô-đun unittest gồm chín ca, sử dụng cơ sở dữ liệu và nhà cung cấp giả lập. Bộ kiểm thử đã được chạy trên working tree cục bộ hiện tại bằng môi trường ảo của dự án (Python 3.10.11): cả 9/9 ca đều đạt trong 0,048 giây. Đây là bằng chứng hồi quy giới hạn cho các luồng đã kiểm tra, không phải đánh giá trên cơ sở dữ liệu hoặc nhà cung cấp thật. Chưa có benchmark mô hình hay kiểm thử tải có thể tái lập. Vì vậy, chương kết quả phân biệt hồ sơ dữ liệu và lần chạy hồi quy đã ghi nhận với các phép đo mô hình, hệ thống còn cần thực hiện.

Từ khóa: trợ lý du lịch, lập lịch trình, OpenStreetMap, PostgreSQL, Flask, kiểm tra xác định, du lịch Việt Nam.

Mục lục

Tóm tắt i

Danh mục chữ viết tắt vii

1. Giới thiệu 1

1.1. Bối cảnh . . . 1
1.2. Phát biểu bài toán . . . 1
1.3. Mục tiêu . . . 2
1.4. Phạm vi và nội dung ngoài phạm vi . . . 2
1.5. Phương pháp và quy ước bằng chứng . . . 3
1.6. Đóng góp và bố cục báo cáo . . . 3

2. Công trình liên quan 4

2.1. Các hệ thống gợi ý du lịch . . . 4
2.1.1. Lọc dựa trên nội dung và lọc cộng tác . . . 4
2.1.2. Lập kế hoạch theo ngữ cảnh và ràng buộc . . . 4
2.2. Mô hình ngôn ngữ lớn và truy xuất làm căn cứ . . . 5
2.3. Điều phối đa tác tử . . . 5
2.4. Tìm kiếm địa lý và lập tuyến . . . 6
2.5. Dữ liệu địa lý mở và chất lượng dữ liệu . . . 6
2.6. So sánh và khoảng trống nghiên cứu . . . 7

3. Tập dữ liệu và tiền xử lý 8

3.1. Nguồn dữ liệu và phạm vi . . . 8
3.2. Lọc ứng viên và chuẩn hóa . . . 8
3.3. Hồ sơ tập dữ liệu đã ghi nhận . . . 9
3.3.1. Vấn đề đối soát trong tệp kê khai . . . 10
3.4. Biểu diễn quan hệ . . . 10
3.5. Tiền xử lý không gian và văn bản . . . 10
3.6. Chất lượng dữ liệu, giấy phép và khả năng tái lập . . . 12
3.7. Tóm tắt . . . 12

4. Cấu hình mô hình **13**

4.1. Kiến trúc ứng dụng . . . 13
4.2. Tìm kiếm du lịch bằng ngôn ngữ tự nhiên . . . 14
4.3. Luồng tạo bản nháp lịch trình . . . 14
4.4. Xác thực và ước lượng tuyến đường . . . 15
4.5. Cấu hình mô hình và nhà cung cấp . . . 16
4.6. Lưu trữ, tài khoản và dịch vụ hỗ trợ . . . 16
4.7. Bảo mật và vận hành . . . 17
4.8. Cấu hình phát triển . . . 17

5. Kết quả thực nghiệm và thảo luận **18**

5.1. Phạm vi đánh giá và bằng chứng hiện có . . . 18
5.2. Kết quả xử lý tập dữ liệu . . . 18
5.3. Thiết kế kiểm thử chức năng trong kho . . . 19
5.4. Hành vi triển khai xác nhận qua mã nguồn . . . 20
5.5. Đề xuất đánh giá truy xuất . . . 20
5.6. Đề xuất đánh giá lịch trình và câu trả lời . . . 21
5.7. Đề xuất đánh giá hiệu năng và độ tin cậy . . . 22
5.8. Thảo luận kết quả hiện tại . . . 22
5.9. Trình tự đánh giá khuyến nghị . . . 23
5.10. Tóm tắt chương . . . 23

Tài liệu tham khảo **24**

A. Từ điển dữ liệu và ghi chú tái lập **26**

A.1. Các trường của bản ghi staging . . . 26
A.2. Danh sách kiểm tra tái lập nhập dữ liệu . . . 27

B. Bề mặt API và ranh giới yêu cầu **28**

B.1. Các tuyến du lịch và lịch trình chính . . . 28
B.2. Xác thực yêu cầu tạo bản nháp . . . 28
B.3. Ranh giới xác nhận bản nháp . . . 29

C. Danh mục kiểm thử hồi quy và đánh giá **30**

C.1. Các ca hồi quy hiện có 30

C.2. Kiểm thử bổ sung nên có trước phát hành 31

C.3. Mẫu ghi nhận đánh giá 31

C.4. Lệnh phát triển cục bộ 31

C.5. Ghi chú kết thúc 32

6. Nhận xét và đánh giá của giảng viên **33**

Danh sách hình

3.1 Luồng lọc và nạp POI được dựng từ mã nguồn 9

4.1 Các luồng ứng dụng đã triển khai và phụ thuộc bên ngoài 13

4.2 Luồng tìm kiếm du lịch bằng ngôn ngữ tự nhiên và bước grounding có điều kiện 14

4.3 Điều phối tạo bản nháp lịch trình và phản hồi xác thực 15

5.1 Luồng bằng chứng đánh giá đề xuất; chưa có benchmark truy xuất, chất lượng lịch trình, căn cứ câu trả lời hoặc hiệu năng 18

Danh sách bảng

2.1 So sánh năng lực trong thiết kế trợ lý du lịch 7

3.1 Số liệu lọc theo lớp được ghi trong bản tổng hợp nhập OSM 9
3.2 Số POI duy nhất theo nhóm danh mục chuẩn hóa 9
3.3 Các thực thể quan hệ chính trong mô hình ứng dụng được khảo sát 11

4.1 Cấu hình nhà cung cấp và cơ sở dữ liệu thể hiện trong mã nguồn 16

5.1 Các ca kiểm thử hồi quy và kết quả của lần chạy cục bộ đã ghi nhận 19

A.1 Từ điển trường của bảng staging OSM 26

B.1 Các endpoint tìm kiếm du lịch và lập lịch chính 28

C.1 Ánh xạ kiểm thử với yêu cầu và kết quả đã ghi nhận 30
C.2 Mẫu ghi nhận cho một lần đánh giá trong tương lai 32
6.1 Phiếu đánh giá đồ án và nhận xét của giảng viên 33

Danh mục chữ viết tắt

Viết tắt	Diễn giải
AI	Trí tuệ nhân tạo
API	Giao diện lập trình ứng dụng
CORS	Chia sẻ tài nguyên giữa các nguồn gốc
GPS	Hệ thống định vị toàn cầu
Haversine	Công thức tính khoảng cách cung lớn trên mặt cầu
HTTP	Giao thức truyền tải siêu văn bản
JSON	Ký pháp đối tượng JavaScript
JWT	Mã thông báo web JSON
LLM	Mô hình ngôn ngữ lớn
MAS	Hệ đa tác tử
OSM	OpenStreetMap
OTP	Mã dùng một lần
POI	Điểm quan tâm
REST	Chuyển trạng thái biểu diễn
SQL	Ngôn ngữ truy vấn có cấu trúc
SMTP	Giao thức truyền thư đơn giản
UI	Giao diện người dùng
VND	Đồng Việt Nam

1. Giới thiệu

1.1. Bối cảnh

Lập kế hoạch du lịch đòi hỏi tổng hợp thông tin thường nằm rải rác trên bản đồ, danh bạ cơ sở kinh doanh, trang đặt dịch vụ, thông tin giao thông công cộng và lời khuyên tại địa phương. Du khách có thể bắt đầu bằng câu hỏi rộng như ăn ở đâu gần một địa danh, rồi bổ sung vị trí, ngày đi, nhịp độ mong muốn hoặc quỹ thời gian. Vì thế, một trợ lý hữu ích cần làm nhiều hơn là tạo văn bản trôi chảy. Hệ thống cần tìm địa điểm có thật, giữ nguyên định danh địa lý, thể hiện rõ phần thông tin còn thiếu và đưa ra lịch trình dễ hiểu, dễ chỉnh sửa.

Bài toán trở nên cụ thể khi danh mục điểm đến được xây dựng từ dữ liệu địa lý do cộng đồng đóng góp. OpenStreetMap là nguồn dữ liệu địa lý cộng tác; mức độ bao phủ và cách gắn thẻ thay đổi theo địa điểm, loại đối tượng [1]. Một bản ghi có thể có tên và tọa độ nhưng thiếu giờ mở cửa, giá vé hoặc địa chỉ đường phố. Nếu hệ thống tự điền những chỗ trống bằng suy đoán nghe có vẻ hợp lý, câu trả lời có thể hữu ích bề ngoài nhưng khiến du khách hiểu sai. Thách thức thiết kế gồm hai phần: dùng bản ghi có cấu trúc làm căn cứ cho địa điểm, đồng thời giữ nguyên sự không chắc chắn khi dữ liệu chưa đầy đủ.

Các mô hình ngôn ngữ lớn (LLM) giúp diễn giải câu hỏi mở và tạo câu trả lời tự nhiên. Kiến trúc Transformer cung cấp nền tảng linh hoạt cho tương tác này [2], [3]. Tuy nhiên, văn phong trôi chảy không bảo đảm điểm tham quan tồn tại, giá hiện tại chính xác hay chuyến đi có thể hoàn thành đúng giờ. Sinh tăng cường truy xuất (RAG) giải quyết một phần vấn đề bằng cách điều kiện hóa nội dung được sinh trên bằng chứng truy xuất được [4], [5]. Bản thân RAG không bảo đảm dữ liệu mới hoặc đầy đủ, không chứng minh kết quả truy xuất phù hợp, cũng không xác thực lịch trình.

Dự án được khảo sát trong báo cáo này, *ChatbotTravel_server* (còn được tài liệu mã nguồn gọi là *VietTravel AI*), là ứng dụng web Python phục vụ tìm kiếm du lịch và quản lý lịch trình. Ứng dụng dùng Flask và Flask-RESTX để cung cấp các điểm cuối HTTP, SQLAlchemy để ánh xạ dữ liệu quan hệ và PostgreSQL làm cơ sở dữ liệu dự kiến. Một giao diện web cục bộ được Flask phục vụ. Quy trình dữ liệu xử lý các điểm quan tâm du lịch từ shapefile OSM Việt Nam rồi nạp bản ghi đã chuẩn hóa vào bảng staging PostgreSQL.

Trong mã nguồn có hai luồng tương tác liên quan nhưng tách biệt. Luồng trò chuyện du lịch phát hiện ngôn ngữ, trích xuất ý định và đặc trưng, tìm dữ liệu staging, rồi dùng nhà cung cấp sinh văn bản đã cấu hình khi khả dụng. Luồng lập lịch nhận yêu cầu có cấu trúc, truy vấn các POI ứng viên, mặc định tạo bản nháp bằng bộ lập lịch xác định, xác thực kết quả và trả về mà chưa lưu. Một thao tác xác nhận riêng kiểm tra bearer token rồi lưu lịch đã xác nhận. Cần phân biệt các luồng này: mã nguồn không triển khai một đồ thị tác tử tự trị duy nhất mà mọi câu hỏi chatbot đều đi qua bộ lập lịch.

1.2. Phát biểu bài toán

Dự án hướng đến câu hỏi kỹ thuật thực tiễn sau:

> Làm thế nào kết hợp giao tiếp hội thoại với dữ liệu điểm đến và quy trình lập lịch có ràng buộc trên nền web, đồng thời nêu rõ nguồn và mức độ chắc chắn của thông tin địa điểm, lịch

trình và tuyến đường?

Câu hỏi này gồm một số vấn đề triển khai:

1. **Căn cứ địa lý.** Câu trả lời được sinh nên dùng POI truy xuất trong phạm vi địa lý của yêu cầu, đồng thời giữ mã nguồn và tọa độ để truy vết gợi ý về danh mục.
2. **Thuộc tính chưa đầy đủ.** Bản ghi cộng đồng không phải lúc nào cũng có giá, đánh giá, thông tin liên hệ hoặc giờ mở cửa. Giá trị chưa biết phải được giữ là chưa biết, thay vì âm thầm thay bằng số liệu bịa đặt.
3. **Tính hợp lệ của lịch.** Lịch trình trong ngày cần có điểm dừng duy nhất, tọa độ hợp lệ, khoảng thời gian được định dạng đúng, không chồng lấn và có tổng thời lượng trong giới hạn hợp lý. Khoảng cách địa lý khác với thời gian đi đường và không nên đánh đồng.
4. **Lưu trữ và danh tính.** Kế hoạch xây dựng từ mã OSM không tự động là bản ghi trong bảng `Attractions` đã được quản lý của ứng dụng. Hai bước tạo nháp và xác nhận phải giữ được khác biệt này, đồng thời xác minh người dùng đã xác thực trước khi ghi dữ liệu.
5. **Khả năng đánh giá.** Báo cáo đồ án cần phân biệt hành vi mã nguồn, tệp kê khai dữ liệu và khai báo kiểm thử với chất lượng mô hình hoặc hiệu năng đã đo. Mỗi loại bằng chứng trả lời một câu hỏi khác nhau.

1.3. Mục tiêu

Các mục tiêu kỹ thuật thể hiện trong mã nguồn đã khảo sát gồm:

1. Cung cấp REST API và giao diện trình duyệt cho câu hỏi du lịch và lập lịch trình.
2. Chuẩn hóa dữ liệu OSM Việt Nam thành POI có tên, loại, tọa độ và thông tin nguồn.
3. Tìm kiếm PostgreSQL theo văn bản hoặc loại; tìm theo bán kính khi có tọa độ.
4. Tạo lịch trình một ngày từ các POI truy xuất được, mặc định dùng bộ lập lịch xác định.
5. Kiểm tra từng bản nháp trong mã ứng dụng và tách riêng lỗi với cảnh báo.
6. Cho phép người dùng đã xác minh xác nhận và lưu bản nháp theo giao dịch.
7. Nêu giới hạn triển khai và đề xuất kế hoạch đánh giá mà không coi thí nghiệm chưa chạy là kết quả.

1.4. Phạm vi và nội dung ngoài phạm vi

Báo cáo bao quát bản chụp kho mã nguồn, gồm điểm vào Flask, application factory, controller, mô-đun dịch vụ, mô hình ORM, migration SQL, tập lệnh nhập dữ liệu, tệp OSM, bộ dữ liệu JSON thử nghiệm và mã nguồn kiểm thử hồi quy. Báo cáo cũng mô tả cấu hình phát triển cục bộ dự kiến trong `docker-compose.dev.yml` và `requirements.txt`.

Phần khảo sát không chứng minh một máy chủ PostgreSQL cụ thể hiện đang chứa đủ số bản ghi trong tệp kê khai. Dự án không kèm tập truy vấn gán nhãn độc lập, hội đồng đánh giá người dùng, nhật ký tải có thể tái lập hay benchmark chất lượng mô hình. Mã nguồn có tích hợp tùy chọn Google Gemini trong bộ lập lịch và một luồng nhà cung cấp riêng cho chatbot; controller lập lịch gọi hàm với `execute_api=False`. Vì thế, mặc định lịch trình được tạo bằng heuristic xác định chứ không phải lời gọi Gemini trực tiếp. Mã

nguồn cũng có phần liên quan đến ChromaDB, nhưng bộ điều phối lịch trình chủ động tắt chức năng này khi tìm trong PostgreSQL.

Do đó, nên xem dự án là nguyên mẫu có giao diện và quy tắc xác thực có thể triển khai, không phải bằng chứng về khả năng bao phủ ở quy mô sản xuất hoặc chất lượng gợi ý đã được đo. Bộ xác thực tĩnh có thể loại dữ liệu sai định dạng nhưng không bảo đảm tuyến đường đúng, giờ hoạt động hiện tại, khả năng tiếp cận, giá cả, độ an toàn hay mức hài lòng.

1.5. Phương pháp và quy ước bằng chứng

Báo cáo được chuẩn bị bằng cách kiểm tra tệp mã nguồn và tài liệu có trong kho. Nhận định về triển khai dẫn tới mô-đun hoặc luồng thực thi cụ thể. Thống kê tập dữ liệu trích từ `data/osm_full_ingestion_summary.json` và được xem là giá trị trong tệp kê khai, không phải số liệu truy vấn cơ sở dữ liệu trực tiếp. Phần kiểm thử mô tả các ca trong `tests/test_multi_agent_flow.py` và lần chạy cục bộ ngày 21/10/2026 bằng môi trường ảo dự án với Python 3.10.11. Cả chín ca đều đạt; lời gọi cơ sở dữ liệu và nhà cung cấp được giả lập. Kết quả chỉ áp dụng cho working tree cục bộ lúc đó, không phải benchmark tích hợp trực tiếp, chất lượng mô hình hay hiệu năng.

Trong toàn báo cáo, bằng chứng được phân thành ba loại:

Đã ghi nhận Giá trị xuất hiện trong tệp dữ liệu, cấu hình, migration hoặc khai báo kiểm thử đã lưu trong kho.

Đã triển khai Hành vi có thể truy vết tới hàm hoặc luồng controller trong mã nguồn.

Chưa đo Hiệu năng, chất lượng hoặc kết quả vận hành chưa có đầu ra chạy thử có thể tái lập trong bản chụp dự án.

Quy ước này đặc biệt quan trọng với Chương 5: tệp kê khai cung cấp thống kê mô tả, mã kiểm thử cho biết ý định kiểm tra, nhưng không loại nào tương đương với benchmark đầu cuối đã được thực hiện.

1.6. Đóng góp và bố cục báo cáo

Đóng góp kỹ thuật chính của kho mã là cấu trúc đầu cuối kết nối chuẩn bị dữ liệu địa lý với ứng dụng du lịch trên web. Bộ nạp lọc và chuẩn hóa các lớp OSM được chọn; mô hình staging quan hệ giữ mã nguồn và thuộc tính gốc; dịch vụ tìm kiếm trả ứng viên từ cơ sở dữ liệu; bộ lập lịch tạo bản nháp theo thời gian; bộ xác thực kiểm tra tập quy tắc cấu trúc và lịch trình. Xác thực tài khoản và lưu lịch trình sau xác nhận là các chức năng bổ sung.

Chương 2 tổng quan hệ gợi ý, cơ chế grounding, điều phối tác tử và lập kế hoạch địa lý liên quan đến thiết kế hiện tại. Chương 3 trình bày bằng chứng OSM, quy tắc nhập, lược đồ và các lưu ý về chất lượng dữ liệu. Chương 4 mô tả kiến trúc vận hành và cấu hình mô hình, nhà cung cấp. Chương 5 chỉ báo cáo bằng chứng có thể truy xuất trong kho, thảo luận hạn chế và đề xuất quy trình đánh giá tiếp theo. Các phụ lục cung cấp từ điển dữ liệu, danh sách API và danh mục kiểm thử, đánh giá.

2. Công trình liên quan

2.1. Các hệ thống gợi ý du lịch

Gợi ý du lịch là bài toán hỗ trợ quyết định phụ thuộc nhiều vào ngữ cảnh. Du khách lựa chọn kết hợp điểm tham quan, ăn uống, lưu trú và phương tiện theo giới hạn về địa điểm, ngày đi, thời gian, chi phí, nhu cầu của nhóm và sở thích cá nhân. Bài toán này khác với xếp hạng một món hàng đơn lẻ vì các lựa chọn tạo thành cả chuyến đi: thứ tự ghé thăm quan trọng, di chuyển tiêu tốn thời gian và nhiều thông tin có thể không chắc chắn. Các khảo sát về hệ gợi ý du lịch xem phương pháp dựa trên nội dung, lọc cộng tác, dựa trên tri thức và nhận biết ngữ cảnh là những nhóm bổ trợ nhau, chứ không phải một phương pháp duy nhất phù hợp mọi trường hợp [6], [7].

2.1.1. Lọc dựa trên nội dung và lọc cộng tác

Lọc dựa trên nội dung xếp hạng địa điểm bằng các thuộc tính tương tự với sở thích đã khai báo hoặc suy ra của người dùng. Nếu POI được biểu diễn bằng vector đặc trưng $\mathbf{x}_i$, còn hồ sơ sở thích là $\mathbf{u}$, một độ tương đồng thường dùng là

$$\cos(\mathbf{u}, \mathbf{x}_i) = \frac{\mathbf{u}^\top \mathbf{x}_i}{\|\mathbf{u}\|_2 \|\mathbf{x}_i\|_2}. \tag{2.1}$$

Phương pháp này có thể gợi ý một địa điểm mới nếu đã có siêu dữ liệu, nhưng chất lượng phụ thuộc vào đặc trưng mô tả đầy đủ và nhất quán. Bản ghi OSM thưa, chỉ có tên, loại và tọa độ, có thể hỗ trợ lọc loại hình và khoảng cách nhưng hầu như không cho biết không khí, khả năng tiếp cận hoặc mức độ phù hợp với một du khách cụ thể. Vì vậy, dự án có danh mục để tìm kiếm, nhưng phần triển khai được khảo sát không có hồ sơ sở thích người dùng đã học hay mô hình gợi ý được huấn luyện.

Lọc cộng tác ước lượng sở thích từ tương tác giữa người dùng và địa điểm. Trong du lịch, tương tác có thể thưa vì mỗi người chỉ đến một điểm đến không thường xuyên, còn người dùng hoặc địa điểm mới có ít lịch sử. PersTour là một ví dụ: phương pháp kết hợp mức độ phổ biến của POI với sở thích người dùng suy ra từ chuỗi ảnh gắn tọa độ, đồng thời cá nhân hóa thời lượng ghé thăm dựa trên thời gian quan sát được tại mỗi POI [8]. Mô hình ORM hiện có ghi nhận tài khoản và lịch trình, nhưng phần mã được kiểm tra không triển khai thuật toán xếp hạng cộng tác hoặc vòng phản hồi đánh giá. Do đó, sẽ không chính xác nếu mô tả hệ thống hiện tại như một bộ gợi ý đã huấn luyện theo nghĩa phân rã ma trận.

2.1.2. Lập kế hoạch theo ngữ cảnh và ràng buộc

Hệ thống nhận biết ngữ cảnh sử dụng các yếu tố như thời gian, vị trí, thời tiết, thành phần đoàn hoặc hoạt động hiện tại bên cạnh sở thích [7]. Lịch trình du lịch còn có các ràng buộc cứng có thể kiểm tra tính khả thi: các lượt ghé thăm không nên chồng lấn, phải nằm trong quỹ giờ cho phép và địa điểm phải thuộc khu vực được hỗ trợ. Có thể thực thi các quy tắc này độc lập với mô hình ngôn ngữ.

Bài toán lập lịch trình có thể mô tả bằng cách chọn và sắp thứ tự tập con $\mathcal{S}$ từ các POI ứng viên. Một hàm mục tiêu tổng quát có thể thưởng cho mức liên quan đến người dùng, đồng thời phạt thời gian và khoảng

cách:

$$\max_{\mathcal{S},\pi} \sum_{i\in\mathcal{S}} u_i - \lambda_d \sum_{j=1}^{|\mathcal{S}|-1} d_{\pi_j,\pi_{j+1}} - \lambda_t \sum_{j=1}^{|\mathcal{S}|} t_{\pi_j}, \tag{2.2}$$

với các ràng buộc về ngân sách, khung thời gian và tổng thời lượng chuyến đi. Trong đó u_i là độ hữu ích của địa điểm, π là thứ tự ghé thăm, d là khoảng cách và t là thời lượng tham quan. Phương trình này mô tả không gian thiết kế hữu ích, không phải hàm tối ưu được bộ lập lịch hiện tại thực thi. Bộ lập lịch xác định hiện dùng một nhóm khung giờ cố định trong ngày và quy tắc ưu tiên ăn uống hoặc tham quan đơn giản khi lựa chọn từ danh sách ứng viên.

2.2. Mô hình ngôn ngữ lớn và truy xuất làm căn cứ

Kiến trúc Transformer dùng cơ chế attention để mô hình hóa quan hệ giữa các phần tử trong chuỗi, là nền tảng của nhiều mô hình ngôn ngữ hiện đại [2]. Mô hình lớn có thể diễn giải nhiều cách diễn đạt và tạo lời giải thích mạch lạc, phù hợp với giao diện hội thoại. Nghiên cứu mô hình ngôn ngữ few-shot cũng cho thấy prompt có thể định hướng hành vi mà không cần khớp tham số riêng cho tác vụ [3]. Tuy vậy, câu trả lời sinh ra không mặc nhiên có căn cứ trong một tập dữ liệu địa phương cập nhật. Mô hình có thể khẳng định một thông tin mà các bản ghi được cung cấp không hề hỗ trợ.

Sinh tăng cường truy xuất kết hợp mô hình sinh với bằng chứng bên ngoài được truy xuất khi trả lời [4]. Dense Passage Retrieval minh họa cách dùng hai bộ mã hóa để tìm đoạn văn bằng biểu diễn vector đã học; Sentence-BERT cung cấp một phương pháp liên quan để tạo embedding câu phục vụ so sánh ngữ nghĩa [9], [10]. Truy xuất sparse như BM25 cung cấp tín hiệu từ vựng bổ trợ, hữu ích với thuật ngữ chính xác [11]. Bộ truy xuất lai có thể kết hợp tương đồng ngữ nghĩa, khớp văn bản, lọc theo loại có cấu trúc và phạm vi địa lý, nhưng mỗi thành phần đều cần được triển khai và đánh giá. Chỉ phụ thuộc vào một cơ sở dữ liệu vector không chứng minh rằng hệ thống đang dùng điểm số lai như vậy.

Kho mã nguồn có thư viện ChromaDB và Sentence-Transformers; `poi_search_tool.py` cũng có một truy vấn Chroma chỉ đọc tùy chọn. Tuy nhiên, bộ điều phối lịch trình gọi tìm kiếm PostgreSQL với `include_chromadb=False`. Luồng truy xuất chính dùng bộ lọc văn bản, loại trong SQL và lọc địa lý khi có tọa độ tâm cùng bán kính. Vì vậy, báo cáo xem luồng lập lịch là truy xuất SQL có cấu trúc, không phải một pipeline RAG vector dense đã triển khai mặc định. Trong luồng trò chuyện, kết quả PostgreSQL được chuyển vào hàm tạo câu trả lời bằng mô hình ngôn ngữ; grounding với Google Search là khả năng riêng, tùy cấu hình và khả năng sẵn sàng của nhà cung cấp. Grounding tìm kiếm không làm siêu dữ liệu OSM đầy đủ hơn và cũng không thay thế đánh giá chuyên biệt theo miền.

2.3. Điều phối đa tác tử

Phương pháp đa tác tử giao các vai trò riêng cho các tác tử phần mềm hoặc mô hình ngôn ngữ có tương tác. Các framework như AutoGen và MetaGPT nghiên cứu cách giao tiếp, chuyên môn hóa vai trò cho tác vụ phức tạp [12], [13]. ReAct xen kẽ suy luận với hành động gọi công cụ [14], còn Reflexion đưa phản hồi từ bước đánh giá kết quả trước đó trở lại quy trình [15]. Những ý tưởng này gợi mở cách chia nhỏ luồng du lịch thành truy xuất, lập lịch, định tuyến và xác thực.

Luồng lịch trình lần lượt gọi các mô-đun ứng dụng. Hàm `orchestrator.py` xác thực yêu cầu, tìm POI trong bảng staging, chuyển ứng viên cho bộ lập lịch, kiểm tra bản nháp và thử lại nếu còn lỗi chặn. Controller API đặt `execute_api=False`, nên endpoint mặc định dùng bộ lập lịch Python xác định. Bên

gọi trực tiếp có thể bật nhánh Gemini tùy chọn trong `planner_agent.py` khi có SDK và thông tin xác thực; API không bật nhánh này.

Các vai trò này là mô-đun phần mềm có đầu vào và đầu ra rõ ràng, không phải các tác tử tự trị chạy đồng thời. Cấu trúc đó giúp kiểm tra từng bước: hệ thống có thể báo thiếu kết quả tìm kiếm, từ chối mã OSM ngoài tập ứng viên và trả cảnh báo riêng. Bộ lập lịch không học từ phản hồi người dùng; bộ xác thực chỉ thực thi những quy tắc đã mã hóa.

2.4. Tìm kiếm địa lý và lập tuyến

Khoảng cách cung lớn giữa hai điểm trên mặt cầu có thể tính bằng công thức Haversine [16]. Với vĩ độ ϕ_1, ϕ_2 và kinh độ λ_1, λ_2 tính theo radian:

$$a = \sin^2\left(\frac{\phi_2 - \phi_1}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right), \tag{2.3}$$

$$d = 2R\,\mathrm{atan2}\left(\sqrt{a}, \sqrt{1-a}\right), \tag{2.4}$$

trong đó R là bán kính Trái Đất. Đây là khoảng cách địa lý, không phải quãng đường trên đường bộ hoặc thời gian di chuyển.

Dự án dùng công thức này theo hai cách liên quan. Hàm tìm kiếm PostgreSQL trước hết lọc bằng hộp giới hạn vĩ độ/kinh độ để giảm số dòng cần xét, sau đó tính khoảng cách Haversine trong SQL để lọc chính xác theo bán kính và sắp xếp gần trước. Bộ trợ giúp định tuyến tính khoảng cách trực tiếp giữa các điểm rồi nhân với hệ số cố định 1,3 để gắn nhãn quãng đường đường bộ ước tính. Thời gian di chuyển được đặt là null và kết quả có cảnh báo; mã nguồn không gọi Mapbox hoặc OSRM trong tuyến lập lịch. Việc nêu rõ mức độ không chắc chắn tốt hơn là tự tạo thời lượng lái xe, nhưng không nên dùng ước lượng này để dẫn đường hoặc khẳng định các khoảng nghỉ giữa chặng là đủ.

Bài toán người bán hàng (TSP) và bài toán định hướng (Orienteering Problem) đưa ra các mô hình chính thức cho việc chọn và sắp thứ tự điểm đến theo giới hạn di chuyển, thời gian [17], [18]. Kho mã nguồn không triển khai bộ giải TSP chính xác hoặc bộ tối ưu mạng lưới đường bộ. Các khung giờ cố định tạo khoảng cách thời gian giữa các lượt ghé thăm; trợ giúp định tuyến chỉ báo khoảng cách ước tính sau khi lập nháp. Bộ tối ưu trong tương lai có thể kết hợp thời lượng theo mạng đường, khung giờ mở cửa và độ hữu ích rõ ràng của người dùng, nhưng cần dữ liệu định tuyến đáng tin cậy và hàm mục tiêu được xác định.

2.5. Dữ liệu địa lý mở và chất lượng dữ liệu

OpenStreetMap là cơ sở dữ liệu bản đồ cộng tác với mức độ bao phủ thuộc tính không đồng đều [1]. Bản trích xuất mở có thể bao phủ diện tích rộng nhưng không phải danh mục du lịch đã biên tập. Tên, thẻ phân loại, địa chỉ và thông tin hoạt động phụ thuộc vào thực tiễn lập bản đồ tại từng nơi. Kho mã ghi nhận bộ shapefile OSM Việt Nam được lấy vào tháng 10/2026 và cấp phép theo Giấy phép Cơ sở dữ liệu Mở. Tập lệnh nhập chọn các lớp liên quan đến du lịch, đồng thời loại bản ghi thiếu tên hoặc có tọa độ ngoài phạm vi Việt Nam.

Làm sạch dữ liệu ảnh hưởng trực tiếp đến hành vi hệ thống. Loại bản ghi không tên tránh trả nhãn trống; giữ giá và giờ mở cửa ở dạng null tránh tạo độ chính xác giả. Khử trùng mã nguồn là cần thiết khi một

đối tượng xuất hiện ở nhiều lớp shapefile. Mặt khác, hộp tọa độ là quy tắc địa lý gần đúng: nó có thể bao gồm điểm ngoài khơi hoặc sát biên giới và không xác nhận một đối tượng thuộc đơn vị hành chính nào. Kết quả chỉ có địa chỉ dạng tọa độ vẫn kém hữu ích hơn địa chỉ đường phố đã xác minh.

2.6. So sánh và khoảng trống nghiên cứu

Bảng 2.1 đối chiếu các nhóm năng lực tổng quát với bằng chứng hiện có trong kho mã. Đây là so sánh mô tả, không phải thí nghiệm đối đầu có kiểm soát.

Bảng 2.1: So sánh năng lực trong thiết kế trợ lý du lịch

Phương pháp	Ưu điểm	Hạn chế trong bối cảnh du lịch
Tìm kiếm từ khóa / SQL	Khớp chính xác, bộ lọc có cấu trúc dễ giải thích	Nhạy với cách diễn đạt và chất lượng siêu dữ liệu; không xếp hạng theo sở thích đã học
Truy xuất vector dense	Có thể khớp các cách hỏi gần nghĩa [9], [10]	Cần đường đi embedding/chỉ mục được cấu hình và tập đánh giá mức liên quan; không có trong luồng lập lịch mặc định
Câu trả lời chỉ từ LLM	Tương tác ngôn ngữ tự nhiên linh hoạt [3]	Có thể nêu thông tin địa điểm không có căn cứ hoặc lỗi thời; không tự áp đặt ràng buộc lịch
Sinh có căn cứ truy xuất	Điều kiện hóa nội dung sinh bằng bằng chứng truy xuất [4]	Chất lượng phụ thuộc độ bao phủ truy xuất, độ mới của nguồn và mức độ dùng đúng bằng chứng
Bộ xác thực ràng buộc	Từ chối lặp lại được các trạng thái sai đã định nghĩa	Không thể xác minh quy tắc chưa được biểu diễn, như giao thông trực tiếp, phép tính ngân sách hoặc mọi định dạng giờ mở cửa
Luồng lập lịch của dự án	Truy xuất ứng viên SQL, lịch xác định, kiểm tra và ranh giới nháp/xác nhận	Chưa đo chất lượng xếp hạng; luồng được khảo sát không tối ưu thời gian đường bộ trực tiếp

Khoảng trống kỹ thuật mà dự án xử lý không phải là các hệ thống hiện hữu chưa có hội thoại hay chức năng địa lý. Thách thức là tích hợp tập dữ liệu địa lý cục bộ, giao diện web, luồng tạo nháp, xác thực tường minh và lưu trữ có xác thực, đồng thời giữ ranh giới giữa giá trị đã biết và ước lượng. Cơ hội nghiên cứu chính là đánh giá liệu tích hợp này có cải thiện tính hữu ích có căn cứ và khả năng đáp ứng ràng buộc lịch trong các truy vấn du lịch Việt Nam thực tế hay không. Bản chụp hiện tại xác nhận có cơ chế triển khai và tình huống kiểm thử ban đầu, nhưng chưa trả lời câu hỏi thực nghiệm đó.

3. Tập dữ liệu và tiền xử lý

3.1. Nguồn dữ liệu và phạm vi

Nguồn địa lý chính là bộ shapefile OSM Việt Nam. README trong kho ghi nhận đây là gói Geofabrik Việt Nam được lấy vào tháng 10/2026, tạo từ OpenStreetMap và phát hành theo Giấy phép Cơ sở dữ liệu Mở (ODbL). Mốc tháng và năm được nêu để mô tả nguồn gốc dữ liệu; nó không khẳng định bản trích xuất vẫn mới nhất hoặc đang được cập nhật.

Tập lệnh nhập `scripts/import_all_osm_poi.py` xử lý ba lớp:

- `gis_osm_pois_free_1`, gồm các điểm có tên như nhà hàng, quán cà phê, khách sạn và đối tượng du lịch;
- `gis_osm_pofw_free_1`, gồm địa điểm thờ tự;
- `gis_osm_natural_free_1`, gồm một số đối tượng tự nhiên có tên.

Các tệp nguồn còn có ranh giới hành chính, đường bộ, đường sắt, sông ngòi, công trình và lớp khác, nhưng bộ nhập POI tổng thể chỉ chọn ba lớp trên. Việc này giảm các bản ghi không liên quan đến danh mục du lịch và tập trung quy trình nhập chính.

Kho mã cũng có bộ dữ liệu `data/osm_vietnam_pilot_sample.json` gồm 15 bản ghi, được dùng trong kiểm thử hồi quy. Bộ này có năm loại quan sát, gồm nhà hàng, quán cà phê, khách sạn, chùa Phật giáo và điểm tham quan núi. Đây là mẫu xác định để kiểm tra hành vi lập lịch ngoại tuyến, không phải mẫu thống kê đại diện cho dữ liệu toàn quốc.

3.2. Lọc ứng viên và chuẩn hóa

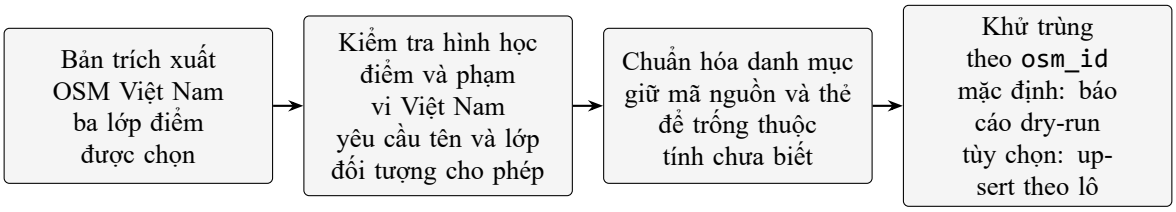
Danh sách cho phép của bộ nhập ánh xạ lớp đối tượng OSM (`fclass`) sang loại hiển thị cho người dùng và nhóm rộng hơn. Danh sách bao gồm ăn uống, lưu trú, điểm tham quan, công viên, bảo tàng, địa điểm thờ tự, đài tưởng niệm và một số đối tượng tự nhiên. Bộ lọc loại bản ghi thiếu tên hiển thị, lớp ngoài danh sách và tọa độ nằm ngoài hộp giới hạn Việt Nam đã cấu hình. Giới hạn triển khai là vĩ độ từ $8.0°$ đến $24.0°$, kinh độ từ $102.0°$ đến $110.0°$. Đây là hình chữ nhật thực dụng, không phải đa giác ranh giới hành chính.

Với bản ghi được giữ lại, bộ nhập bảo toàn lớp nguồn, mã OSM, lớp OSM và mã số, tên, tọa độ, thuộc tính gốc cùng thẻ. Hệ thống tạo loại chuẩn hóa và mô tả tóm tắt. Nếu không có địa chỉ dùng được, giá trị dự phòng được cấu hình là câu tiếng Việt rõ nghĩa: *Chưa có địa chỉ cụ thể; dùng tọa độ OpenStreetMap*. Giá, đánh giá, giờ mở cửa, số điện thoại và URL hình ảnh chưa biết được giữ là null. Cách làm này giữ nguồn gốc dữ liệu và tránh biến thông tin thiếu thành dữ kiện bịa đặt.

Lệnh nhập mặc định chạy thử phân tích, chưa ghi dữ liệu; tùy chọn `--apply` mới bật ghi vào cơ sở dữ liệu. Kích thước lô có thể cấu hình, mặc định được tài liệu ghi là 1.000. Bảng staging áp đặt tính duy nhất cho mã OSM; bộ nhập dùng upsert theo `osm_id` để cập nhật bản ghi nguồn đã có hoặc chèn bản

ghi mới. Migration khai báo cột JSONB cho thẻ, bí danh và thuộc tính gốc, nhờ đó chi tiết nguồn có thể được lưu mà không cần tách mọi thuộc tính OSM thành một cột quan hệ riêng.

Có thể tóm tắt luồng chuyển đổi thành chuỗi bộ lọc giữ lại nguồn gốc dữ liệu. Hình 3.1 thể hiện cấu hình mặc định của bộ nhập: chỉ ghi vào cơ sở dữ liệu khi có tùy chọn `--apply`.



Hình 3.1: Luồng lọc và nạp POI được dựng từ mã nguồn

3.3. Hồ sơ tập dữ liệu đã ghi nhận

Tệp đã lưu trong kho `data/osm_full_ingestion_summary.json` là nguồn cho hồ sơ mô tả trong mục này. Tệp ghi 30.941 ứng viên sau lọc và 30.839 định danh duy nhất. Số liệu theo lớp và số lượng bị loại được chép lại trong Bảng 3.1. Đây là giá trị trong bản tổng hợp xử lý, không phải kết quả truy vấn trực tiếp cơ sở dữ liệu.

Bảng 3.1: Số liệu lọc theo lớp được ghi trong bản tổng hợp nhập OSM

Lớp nguồn	Đầu vào	Ứng viên	Thiếu tên	Loại	Tọa độ
POI OSM	75.037	28.482	23.862	22.678	15
Địa điểm thờ tự	1.375	1.127	248	0	0
Đối tượng tự nhiên	34.009	1.332	32.596	81	0
Tổng	**110.421**	**30.941**	**56.706**	**22.759**	**15**

Tổng theo nhóm danh mục trong cùng tệp là 30.839, như Bảng 3.2. Dữ liệu tập trung chủ yếu vào ăn uống và lưu trú. Số bản ghi nhiều trong một nhóm không đồng nghĩa chất lượng hoặc độ đầy đủ cao; đó chỉ là số lượng định danh được phân loại trong bản tổng hợp.

Bảng 3.2: Số POI duy nhất theo nhóm danh mục chuẩn hóa

Nhóm danh mục	Bản ghi	Tỷ lệ
Ăn uống	18.317	59,40%
Lưu trú	6.454	20,92%
Du lịch	2.082	6,75%
Thiên nhiên	1.311	4,25%
Địa điểm tâm linh	1.129	3,66%
Văn hóa	789	2,56%
Lịch sử	643	2,08%
Giải trí	114	0,37%
Tổng	**30.839**	**100,00%**

3.3.1. Vấn đề đối soát trong tệp kê khai

Bản tổng hợp ghi 30.941 ứng viên và 30.839 định danh duy nhất, chênh lệch 102 dòng. Một trường riêng lại báo 100 định danh trùng, gồm 27 trùng giữa các lớp và 73 trùng trong cùng lớp. Các trường này không khớp về số học: tổng trùng được nêu thấp hơn hai so với chênh lệch suy ra từ tổng ứng viên và tổng duy nhất. Tổng nhóm danh mục khớp với 30.839 duy nhất. Báo cáo ghi nhận sai khác thay vì tự ý điều chỉnh; cần chạy lại bộ nhập kèm kiểm tra từng định danh trước khi dùng các con số này làm tổng chấp nhận cuối cùng của cơ sở dữ liệu.

Bản tổng hợp còn có khối phân tích chồng lấp ghi 30.839 bản ghi staging hiện có và 30.839 bản cập nhật, đồng thời không có bản chèn hoặc xóa. Khối này thể hiện trạng thái tại thời điểm quy trình tạo bản tổng hợp, không xác nhận độc lập trạng thái hiện tại của PostgreSQL đang chạy. Bản chụp kho không có bản sao cơ sở dữ liệu hoặc kết quả truy vấn để xác minh bảng staging hiện còn số dòng đó. Khối `core_tables` liệt kê 0 bản ghi cho Attractions, Users và Itineraries trong phân tích của tệp kê khai; không nên hiểu các số này là thống kê cơ sở dữ liệu đang vận hành hôm nay.

3.4. Biểu diễn quan hệ

Ứng dụng dùng mô hình SQLAlchemy cho tài khoản, hội thoại, lịch trình và địa điểm. Bảng 3.3 tóm tắt các thực thể chính nhìn thấy trong ORM và migration staging. Các bảng ứng dụng theo kiểu sản phẩm dùng tên hỗn hợp chữ hoa-thường như `Users`, `Attractions` và `ItineraryItems`, trong khi bảng staging OSM có tên chữ thường `osm_attractions_staging`.

Việc tách `osm_attractions_staging` khỏi `Attractions` có ý nghĩa kiến trúc. Bản staging được định danh bằng `osm_id`; mục lịch trình tham chiếu `Attractions.id` của ứng dụng. Dịch vụ xác nhận vì vậy phải tìm hoặc tạo bản ghi attraction tương ứng và kiểm tra lại bản ghi OSM trước khi lưu. Nếu coi mã OSM là khóa của attraction lõi thì tính toàn vẹn tham chiếu sẽ bị phá vỡ.

3.5. Tiền xử lý không gian và văn bản

Bộ nhập chuẩn hóa lớp đối tượng thành danh mục, giữ mã nguồn và kiểm tra phạm vi tọa độ trước khi nạp dữ liệu. Với tìm kiếm theo bán kính, `poi_search_tool.py` dùng hai bước. Trước tiên, hàm tính hình chữ nhật bao quanh tâm tìm kiếm để giảm số dòng ứng viên. Sau đó, hàm tính khoảng cách Haversine trong SQL, loại kết quả ngoài bán kính yêu cầu, sắp theo khoảng cách tăng dần và áp dụng giới hạn kết quả sau khi đã sắp xếp. Nhờ vậy, các kết quả đầu không được chọn chỉ theo thứ tự ID trong bảng khi truy vấn có yếu tố không gian.

Phép tính dùng bán kính Trái Đất trung bình 6.371 km. Với tâm (ϕ_c, λ_c) và bản ghi (ϕ, λ), tất cả góc được đổi sang radian. Khoảng cách trực tiếp:

$$d = 2R \arcsin \sqrt{\sin^2\left(\frac{\phi - \phi_c}{2}\right) + \cos(\phi_c)\cos(\phi)\sin^2\left(\frac{\lambda - \lambda_c}{2}\right)}. \tag{3.1}$$

Độ rộng kinh độ của hộp giới hạn được điều chỉnh theo cosin vĩ độ tâm và có chặn dưới trong mã nguồn. Đây là bước tối ưu truy vấn; điều kiện Haversine vẫn là bộ lọc cuối cùng theo bán kính.

Tìm kiếm văn bản dùng các điều kiện `LIKE` đã chuẩn hóa chữ hoa-thường trên tên, danh mục hoặc lớp đối tượng; tham số SQL được truyền riêng. Đây là khớp từ vựng, không phải mô hình tìm kiếm ngữ

Bảng 3.3: Các thực thể quan hệ chính trong mô hình ứng dụng được khảo sát

Thực thể	Định danh và liên kết	Mục đích và trường đáng chú ý
Users	`user_id` kiểu số nguyên; cha của hội thoại và lịch trình	Email, mã băm mật khẩu, ngôn ngữ ưu tiên, trạng thái xác minh
Attractions	`id` kiểu chuỗi; được mục lịch trình tham chiếu	Chi tiết địa điểm đã biên tập, tọa độ, loại, giá và giờ tùy chọn
Conversations	`conversation_id` kiểu số nguyên; khóa ngoại tới Users	Thời gian bắt đầu/kết thúc, ngôn ngữ nguồn, tiêu đề
Messages	`message_id` kiểu số nguyên; khóa ngoại tới Conversations	Người gửi, văn bản, bản dịch, URL giọng nói, danh sách địa điểm JSON
Itineraries	`id` kiểu số nguyên; khóa ngoại tới Users	Ngày chọn, tiêu đề, ghi chú, cờ xóa mềm
ItineraryItems	`id` kiểu số nguyên; liên kết Itineraries và Attractions	Thời gian, thời lượng, ghi chú, thứ tự ghé thăm
Notifications	`id` kiểu số nguyên; liên kết Users và Itineraries	Nội dung nhắc, giờ gửi, trạng thái đã gửi/đọc/xóa
OTP	`id` kiểu số nguyên; khóa theo email và mục đích trong dịch vụ	Mã xác minh hoặc đặt lại mật khẩu và thời hạn
OSM staging	ID dòng số nguyên; `osm_id` duy nhất	Nguồn/lớp OSM, loại chuẩn hóa, GPS, thuộc tính JSONB

nghĩa tiếng Việt. Dịch vụ chatbot có thêm hàm trích xuất ý định và đặc trưng, nhưng API lập lịch dùng các trường yêu cầu đã chuẩn hóa riêng. Không nên mặc định rằng hệ thống có chuẩn hóa dấu, mở rộng truy vấn hay bí danh tên đa ngôn ngữ nếu luồng cụ thể không triển khai những chức năng đó.

3.6. Chất lượng dữ liệu, giấy phép và khả năng tái lập

Mô hình dữ liệu chủ động cho phép giá, đánh giá, giờ mở cửa, số điện thoại và ảnh ở dạng chưa biết. Điều này trung thực về tri thức nhưng giới hạn mức cá nhân hóa của lịch trình. Nếu giờ mở cửa là null, bộ xác thực phát cảnh báo và đề nghị du khách tự kiểm tra. Hàm chỉ phân tích định dạng đơn giản `HH:MM-HH:MM`; biểu thức OSM phức tạp được xem là cần xác nhận thủ công. Địa chỉ thiếu được thay bằng chuỗi dự phòng, không phải địa danh được dịch ngược từ tọa độ.

Khi phân phối hoặc cung cấp công khai tập dữ liệu OSM, cần giữ thông tin ghi công và tuân thủ giấy phép cơ sở dữ liệu. README nguồn nêu ODbL và ngày của dữ liệu, nhưng giao diện web cùng lược đồ phản hồi API được khảo sát không cho thấy trường ghi công nhất quán cho từng địa điểm. Bản triển khai cần hiển thị ghi công theo yêu cầu và thông báo các thay đổi đối với cơ sở dữ liệu, đồng thời lưu ngày trích xuất cùng phiên bản bộ nhập để có thể tái lập.

Dự án cung cấp đủ tạo tác cục bộ để mô tả thiết kế xử lý dữ liệu: shapefile gốc, mã nhập, DDL migration, tệp tổng hợp dữ liệu và bộ mẫu kiểm thử nhỏ. Một bản phát hành tập dữ liệu có thể tái lập nên ghi thêm tham số dòng lệnh, checksum tệp nguồn, phiên bản Python và các gói, số dòng trong cơ sở dữ liệu, cùng báo cáo đối soát trùng lặp đầy đủ. Các thông tin này giúp phân biệt thay đổi từ bản trích xuất nguồn với thay đổi do mã chuyển đổi.

3.7. Tóm tắt

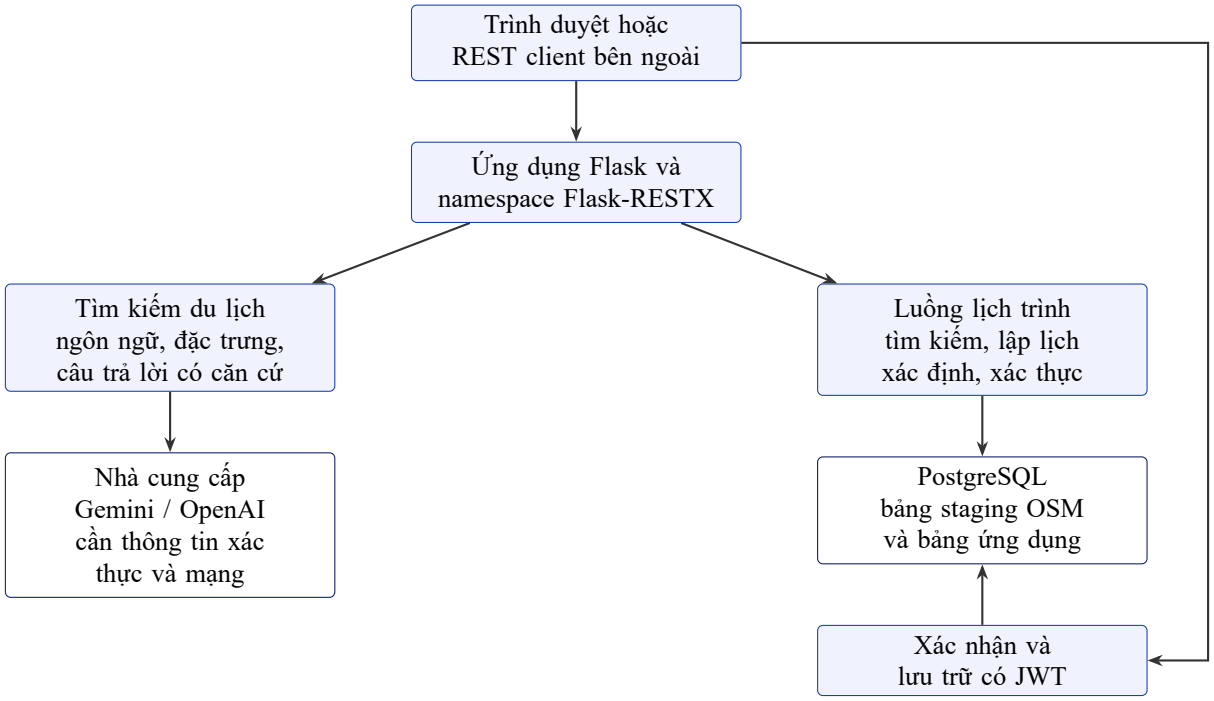
Kho mã có luồng nhập OSM ở quy mô toàn quốc và lược đồ staging được thiết kế để giữ định danh nguồn cùng thông tin về thuộc tính còn thiếu. Bản tổng hợp xử lý đã lưu mô tả khoảng ba mươi nghìn POI được phân loại; nhóm ăn uống và lưu trú chiếm phần lớn. Tệp này cũng có sai khác số học hai bản ghi trong phần thống kê trùng lặp. Bộ dữ liệu mẫu và tệp kê khai không chứng minh rằng toàn bộ bản ghi hiện đang có trong cơ sở dữ liệu đã cấu hình. Những lưu ý này là cơ sở để đọc thiết kế và đánh giá ở các chương sau: độ bao phủ và chất lượng dữ liệu là hai thuộc tính riêng, cần được kiểm tra trực tiếp.

4. Cấu hình mô hình

4.1. Kiến trúc ứng dụng

Ứng dụng là một dịch vụ Flask duy nhất, có giao diện trình duyệt và REST API. `main.py` nạp biến môi trường, khởi tạo ứng dụng qua `create_app()`, chạy bộ lập lịch nhắc lịch trình và khởi động máy chủ phát triển tại `127.0.0.1:5000`. Application factory khởi tạo Flask-CORS, Flask-Mail, SQLAlchemy và Flask-RESTX; phục vụ trang `index.html` ở đường dẫn gốc; đồng thời đăng ký các namespace cho xác thực, chat, tìm kiếm du lịch, lịch trình, bản đồ, thông báo và thao tác dữ liệu vị trí cũ. Tài liệu API tương tác được cung cấp tại `/docs`.

Hình 4.1 cho thấy các kết nối mức cao được mã nguồn hỗ trợ. Sơ đồ tách riêng luồng tìm kiếm du lịch có dùng mô hình ngôn ngữ khỏi luồng lập lịch xác định.



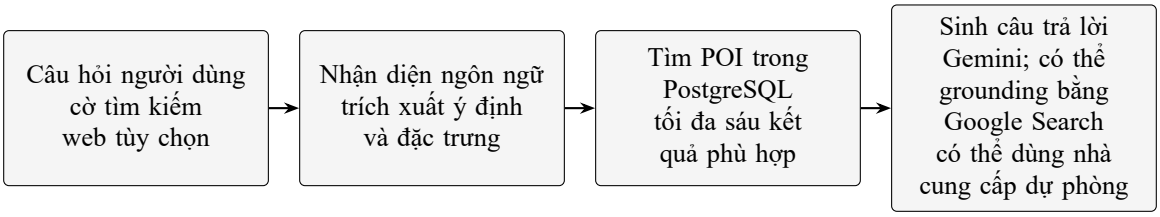
Hình 4.1: Các luồng ứng dụng đã triển khai và phụ thuộc bên ngoài

Giao diện trình duyệt viết bằng HTML, CSS và JavaScript trong `src/templates` và `src/static`. Giao diện gọi API cho câu hỏi, kết quả địa điểm, tài khoản và thao tác lịch trình. Controller chuyển yêu cầu HTTP thành lời gọi dịch vụ; lớp dịch vụ chứa logic nhà cung cấp, tìm kiếm, xác thực, thông báo và lập lịch; mô hình ánh xạ dữ liệu quan hệ sang đối tượng SQLAlchemy. Cách tổ chức theo lớp giúp nhận biết các luồng yêu cầu chính, dù kho mã vẫn có một số mô-đun cũ cùng tồn tại với thành phần lập lịch mới hơn.

4.2. Tìm kiếm du lịch bằng ngôn ngữ tự nhiên

Endpoint `/api/travel-chatbot/search` nhận một câu hỏi và cờ tùy chọn `enable_web_search`. Controller từ chối yêu cầu không có câu hỏi, gọi nhận diện ngôn ngữ, rồi chuyển câu hỏi tới dịch vụ trích xuất ý định và đặc trưng có cấu trúc. Mã nguồn có xử lý qua nhà cung cấp và phương án dự phòng, cùng một số luật trích xuất đặc trưng. Dịch vụ tìm kiếm truy vấn dữ liệu staging PostgreSQL để lấy tối đa sáu địa điểm phù hợp. Hàm sinh phản hồi nhận câu hỏi gốc, bản ghi truy xuất, đặc trưng, ngôn ngữ phát hiện và tùy chọn tìm kiếm web. Kết quả có thể gồm câu trả lời, ngôn ngữ, POI, hoạt động gợi ý, câu hỏi tiếp theo và đặc trưng đã trích xuất.

Hình 4.2 tóm tắt luồng chatbot được triển khai. Grounding bằng Google Search là tính năng tùy chọn của nhà cung cấp; sơ đồ không hàm ý mọi yêu cầu đều thực hiện tìm kiếm bên ngoài.



Hình 4.2: Luồng tìm kiếm du lịch bằng ngôn ngữ tự nhiên và bước grounding có điều kiện

Dịch vụ chat là nơi dễ thấy nhất luồng sinh ngôn ngữ dựa trên nhà cung cấp. `travel_chatbot_service.py` có lời gọi REST Gemini, thử dự phòng giữa các mô hình, tùy chọn grounding bằng Google Search và phương án dự phòng OpenAI cũ. Luồng thực tế phụ thuộc vào khóa, mạng, phản hồi nhà cung cấp và chế độ yêu cầu. Siêu dữ liệu grounding có thể chứa truy vấn web và URL nguồn. Cần xem đây là các tích hợp có điều kiện: bản thân kho mã không chứng minh rằng thông tin xác thực đã được khai báo hoặc lời gọi bên ngoài sẽ thành công trong môi trường triển khai.

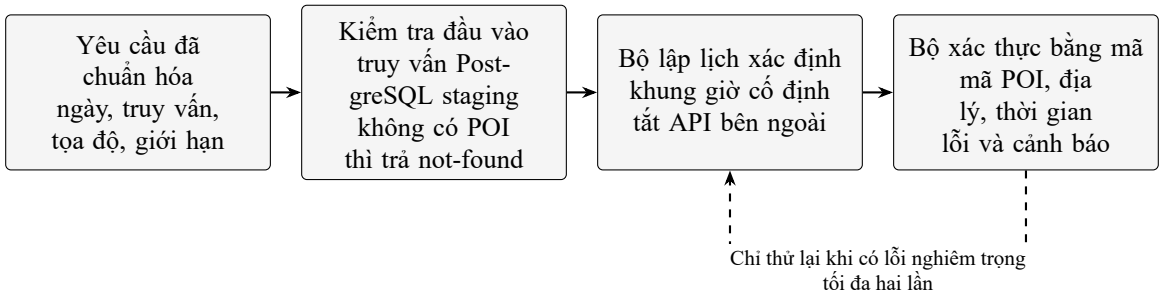
Trợ giúp tìm kiếm chatbot và công cụ tìm kiếm cho lịch trình không thể thay thế lẫn nhau. Luồng chatbot dùng đặc trưng trích xuất từ hội thoại và giới hạn số kết quả phù hợp câu trả lời chat. Luồng lập lịch nhận tham số đã chuẩn hóa tường minh rồi tìm ứng viên trong `osm_attractions_staging`. Câu hỏi được chatbot trả lời không tự động được lưu thành kế hoạch du lịch.

4.3. Luồng tạo bản nháp lịch trình

API lịch trình cung cấp `/api/itinerary/plan-draft` cùng bí danh `/api/itinerary/multi-agent/plan-draft`. Controller yêu cầu ngày chọn theo định dạng ISO và có từ khóa tìm kiếm hoặc tọa độ. Nếu chỉ gửi tên khu vực hoặc địa danh mà không có tọa độ, controller trả lỗi đầu vào vì luồng này không geocoding. Khi có tọa độ, vĩ độ và kinh độ phải nằm trong hộp giới hạn Việt Nam; bán kính phải từ 0,1 đến 100 km. Bán kính mặc định cho yêu cầu có tâm tọa độ là 5 km. Yêu cầu cũng nhận danh mục, sở thích, nội dung ngân sách và số điểm dừng tối đa.

Sau khi chuẩn hóa đầu vào, controller gọi `orchestrate_draft_itinerary()` với `execute_api=False` và tối đa hai lần thử lại. Trình tự gồm xác thực ngày, cặp tọa độ, bán kính, giới hạn kết quả và số điểm dừng; truy vấn staging; trả trạng thái không tìm thấy nếu không có POI; tạo bản nháp; kiểm tra bản nháp theo danh sách ứng viên gốc, địa lý và thời gian; thử lại nếu còn lỗi nghiêm trọng; rồi trả lịch trình cùng báo cáo xác thực, các chặng, cảnh báo, số ứng viên và số vòng lặp. Chỉ cảnh báo không kích hoạt thử lại. Kết quả từ endpoint được đánh dấu là chưa lưu.

Hình 4.3 mô tả luồng bản nháp bằng các mô-đun tuần tự. Lỗi xác thực có thể kích hoạt thử lại; riêng cảnh báo thì không. Endpoint trả bản nháp nhưng chưa lưu.



Hình 4.3: Điều phối tạo bản nháp lịch trình và phản hồi xác thực

Bộ lập lịch mặc định chia ứng viên thành nhóm ăn uống và nhóm khác, sau đó phân vào chuỗi khung giờ cố định bắt đầu lúc 08:30, 10:30, 12:30, 14:45 và 18:00. Thời lượng mỗi lượt từ 90 đến 120 phút. Theo nhãn khung giờ, bộ lập lịch cố gắng xen kẽ ăn uống và tham quan, tránh dùng trùng `osm_id`, đồng thời giới hạn số điểm theo yêu cầu và số ứng viên hiện có. Đây là heuristic cơ sở, không chứng minh lịch trình tối ưu toàn cục hoặc cá nhân hóa.

Bộ tạo prompt và nhánh Gemini tùy chọn có trong `planner_agent.py`; nhánh này yêu cầu JSON có cấu trúc và cấu hình nhiệt độ thấp. Tuy nhiên, endpoint truyền `execute_api=False`, nên đường đi API mặc định dùng `deterministic_planner_v1` bất kể có khóa Gemini hay không. Các vai trò trong điều phối là mô-đun phần mềm tuần tự có đầu vào và đầu ra, không phải một tập tác tử tự trị chạy đồng thời. Cách tổ chức này giúp kiểm tra riêng từng bước và từ chối mã OSM ngoài tập ứng viên, nhưng heuristic không học từ phản hồi người dùng và bộ xác thực chỉ kiểm tra những quy tắc đã mã hóa.

4.4. Xác thực và ước lượng tuyến đường

Bộ xác thực tách lỗi chặn bản nháp khỏi cảnh báo. Hàm kiểm tra mỗi điểm dừng có mã OSM thuộc tập ứng viên và không lặp điểm. Tọa độ phải là số, nằm trong cùng giới hạn Việt Nam rộng được bộ lập lịch sử dụng. Hàm kiểm tra định dạng giờ `HH:MM`, yêu cầu giờ kết thúc sau giờ bắt đầu, đối chiếu thời lượng khai báo với khoảng thời gian, loại các lượt chồng lấn và giới hạn tổng khoảng thời gian trong ngày là 12 giờ.

Kiểm tra giờ mở cửa chỉ thực hiện khi có thể. Khoảng giờ đơn giản được đối chiếu với khoảng ghé thăm. Giá trị bị thiếu hoặc giá trị phức tạp mà bộ phân tích đơn giản không đọc được tạo cảnh báo để kiểm tra thủ công. Bộ xác thực không tuyên bố phân tích đầy đủ ngữ pháp opening-hours của OSM.

Trợ giúp định tuyến tính khoảng cách Haversine giữa hai điểm liên tiếp rồi nhân với 1,3. Kết quả được gắn nhãn ước tính, còn `travel_time_minutes` là null. Hàm đọc token Mapbox từ môi trường có tồn tại, nhưng tuyến lập lịch được khảo sát không gọi mạng tới Mapbox hoặc OSRM. Khi không có thời lượng lái xe thật, hệ thống thông báo chưa xác định được khoảng thời gian di chuyển thay vì trình bày ước lượng như thời lượng tuyến đường. Khoảng cách gần đúng chỉ là chỉ báo thô về độ tách biệt địa lý.

Các bước kiểm tra không áp đặt ngân sách được yêu cầu dù yêu cầu có thể mang giá trị này. Hệ thống cũng không tối ưu thứ tự dừng theo mạng đường, tính giờ mở cửa theo lịch tuần, xét lịch giao thông công cộng hay xác thực thời tiết và sự kiện. Đây là giới hạn phạm vi, không phải kiểm thử thất bại.

4.5. Cấu hình mô hình và nhà cung cấp

Dự án có cấu hình nhà cung cấp riêng cho câu trả lời hội thoại và bộ lập lịch. Bảng 4.1 tóm tắt các điểm cấu hình nhìn thấy trong mã nguồn. Thông tin xác thực được đọc từ biến môi trường và không được sao chép vào báo cáo.

Bảng 4.1: Cấu hình nhà cung cấp và cơ sở dữ liệu thể hiện trong mã nguồn

Khu vực	Cấu hình	Hành vi quan sát được
Trả lời chat	`GEMINI_API_KEYS`, `GEMINI_API_KEY`, `GEMINI_MODEL`	REST Gemini thử một nhóm mô hình; có thể grounding Google Search nếu được yêu cầu và hỗ trợ.
Dự phòng chat	`OPENAI_API_KEY`	Dịch vụ chat du lịch có lời gọi OpenAI cũ làm phương án dự phòng.
Bộ lập lịch nháp	`GEMINI_API_KEYS`, `GEMINI_MODEL`	Có nhánh SDK Gemini nhưng endpoint HTTP tạo nháp tắt thực thi API bên ngoài.
Định tuyến bản đồ	`MAPBOX_ACCESS_TOKEN` hoặc `MAPBOX_TOKEN`	Có hàm lấy token; tuyến lịch trình hiện trả ước lượng Haversine.
Cơ sở dữ liệu	`DATABASE_POSTGRESQL_URL`	SQLAlchemy dùng PostgreSQL khi có URL cơ sở dữ liệu.
Email	`MAIL_SERVER`, `MAIL_PORT`, TLS và cấu hình tài khoản	Flask-Mail gửi OTP và email nhắc lịch nếu cấu hình SMTP khả dụng.
Xác thực	`SECRET_KEY`	Dùng để ký và xác minh JWT của ứng dụng.

Bộ lập lịch Gemini tùy chọn yêu cầu JSON có cấu trúc và dùng nhiệt độ 0,2. Chuỗi mô hình mặc định là `gemini-3-flash-preview`, nhưng bên gọi có thể ghi đè bằng cấu hình. Nếu SDK hoặc lời gọi nhà cung cấp thất bại, bộ lập lịch quay về hàm xác định. Tuyến HTTP mặc định tránh phụ thuộc này vì không bật nhánh API.

Dịch vụ tìm kiếm du lịch tạo yêu cầu REST tới các mô hình Gemini với nhiệt độ và giới hạn token đầu ra đã cấu hình, thử lại qua danh sách mô hình và có thể yêu cầu công cụ Google Search khi sinh nội dung không phải JSON. Nếu tuyến Gemini không khả dụng, dịch vụ có thể dùng lời gọi OpenAI cũ. Các cấu hình này không thiết lập việc tạo lịch trình vì endpoint bộ lập lịch mặc định chủ động chạy ngoại tuyến.

4.6. Lưu trữ, tài khoản và dịch vụ hỗ trợ

Tạo bản nháp và lưu trữ là hai thao tác riêng. Tuyến tạo nháp trả đối tượng trong bộ nhớ và gắn `is_saved=False`. Endpoint xác nhận có tại `/api/itinerary/confirm` và `/api/itinerary/multi-agent/confirm`. Endpoint yêu cầu `draft_id`, ngày chọn và danh sách mục không rỗng, sau đó lấy danh tính tài khoản từ JWT Bearer thay vì tin mã người dùng trong thân yêu cầu. Máy chủ xác minh chữ ký và hạn token, tra người dùng rồi từ chối tài khoản chưa xác thực email. Dịch vụ xác nhận kiểm tra yêu cầu theo dữ liệu staging và ghi lịch ứng dụng qua mô hình quan hệ.

Các endpoint tài khoản gồm đăng ký, xác minh email bằng mã một lần, đăng nhập, các bước đặt lại mật khẩu và đổi tên người dùng. Mô hình người dùng lưu mã băm mật khẩu thay vì mật khẩu thuần. Mô hình

chat lưu hội thoại và tin nhắn; thực thể lịch trình liên kết người dùng, ngày đi và các lượt ghé attraction có thứ tự. Namespace thông báo hỗ trợ truy vấn, đánh dấu đã đọc và xóa.

Application factory gọi `db.create_all()` trong app context, nhưng bắt lỗi kết nối và ghi cảnh báo. Kho cũng có migration SQL cho bảng staging OSM và thông báo. Cách kết hợp này hỗ trợ phát triển cục bộ, nhưng không thay thế framework migration có phiên bản cho mọi thay đổi mô hình; triển khai cần áp dụng và xác minh migration tường minh.

Bộ lập lịch trong `scheduler_service.py` được khởi chạy khi chạy `main.py`. Theo chu kỳ, nó kiểm tra lịch trình sắp tới rồi gọi dịch vụ email. Điểm vào phát triển bật Flask debug và chỉ bind vào localhost, phù hợp cho phát triển cục bộ nhưng cần thay đổi khi triển khai sản xuất. Docker Compose cung cấp container phát triển tương thích PostgreSQL dùng image pgvector, tuy nhiên luồng lập lịch ở đây dùng cột quan hệ, không cần thao tác phần mở rộng vector.

4.7. Bảo mật và vận hành

Dự án đọc thông tin kết nối cơ sở dữ liệu, thư, mô hình, bản đồ và khóa ký từ cấu hình môi trường. Báo cáo không để lộ giá trị trong tệp `.env` cục bộ. Khi triển khai cần giữ tệp môi trường ngoài quản lý mã nguồn, xoay vòng thông tin xác thực đã bị lộ, dùng khóa ký mạnh và giới hạn quyền truy cập cơ sở dữ liệu. Tích hợp HTTP với nhà cung cấp kéo theo chi phí, giới hạn tốc độ, tính sẵn sàng và việc truyền dữ liệu. Câu hỏi người dùng có thể chứa ngữ cảnh cá nhân; cần công khai và giảm thiểu lời gọi nhà cung cấp theo yêu cầu riêng tư của nơi triển khai.

Luồng xác nhận có ranh giới danh tính hữu ích: máy chủ suy ra người dùng từ token đã ký, không chấp nhận `user_id` trong JSON làm căn cứ. Dù vậy, cần đánh giá riêng hạn dùng, giới hạn tốc độ, chống dò đoán và truyền tải an toàn đối với token, OTP, đăng ký và đặt lại mật khẩu trước khi mở dịch vụ công khai. Trạng thái kết nối cơ sở dữ liệu, migration, nhà cung cấp và SMTP nên được theo dõi bằng log có cấu trúc, không ghi bí mật hoặc nội dung cá nhân không cần thiết.

SQLAlchemy dùng tham số cho giá trị truy vấn lịch trình. Migration staging khai báo ràng buộc duy nhất trên `osm_id` và chỉ mục cho tọa độ, danh mục, lớp đối tượng, lớp nguồn và trạng thái nhập. Các cấu trúc này hỗ trợ truy xuất và nhập lặp an toàn. Hiệu năng quy mô lớn chưa được đo trong bản chụp kho; cần kiểm thử với cơ sở dữ liệu được cấp phát và ghi lại phần cứng cùng cấu hình.

4.8. Cấu hình phát triển

Thiết lập cục bộ được kho mã mô tả gồm tạo môi trường ảo Python, cài `requirements.txt`, khởi chạy cơ sở dữ liệu phát triển bằng `docker compose -f docker-compose.dev.yml up -d`, khai báo biến môi trường cần thiết, áp dụng migration SQL phù hợp, nhập POI với cờ `--apply` tường minh rồi chạy `python main.py`. Giao diện phát triển ở đường dẫn gốc và tài liệu Swagger tại `/docs`.

Do bộ nhập mặc định chạy thử và endpoint tạo nháp mặc định hoạt động xác định, ngoại tuyến, có thể phát triển luồng xử lý tập dữ liệu và lập lịch cốt lõi mà không cần gọi API mô hình. Chức năng dựa trên cơ sở dữ liệu vẫn cần PostgreSQL truy cập được và có bảng staging. Sinh câu trả lời chatbot tùy chọn, SMTP và dịch vụ bản đồ cần thông tin xác thực cùng quyền truy cập mạng tương ứng.

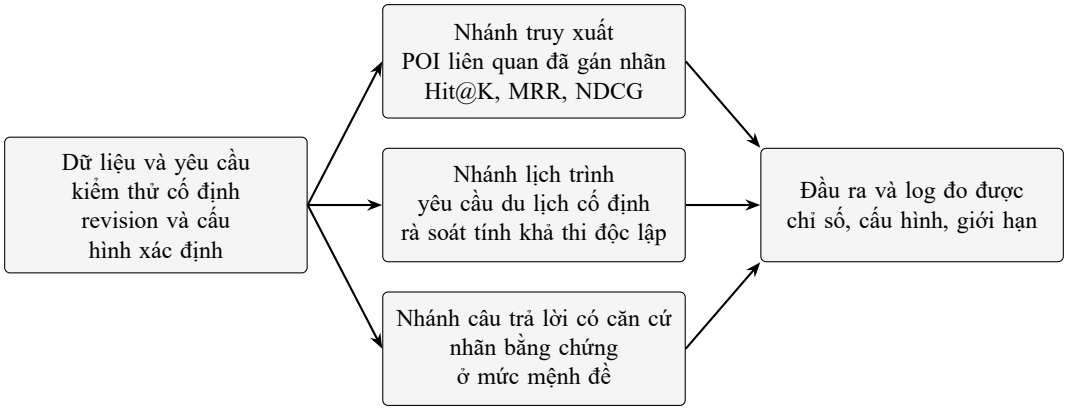
5. Kết quả thực nghiệm và thảo luận

5.1. Phạm vi đánh giá và bằng chứng hiện có

Đánh giá định lượng cần dựa trên tập dữ liệu cố định, truy vấn được gán nhãn, cấu hình hệ thống xác định và đầu ra chạy thử được lưu lại. Bản chụp kho mã có bản tổng hợp xử lý dữ liệu, bộ mẫu nhỏ, mã triển khai và mô-đun kiểm thử. Bộ unittest gồm chín ca đã được chạy trên working tree cục bộ hiện tại; kết quả được ghi dưới đây. Dự án vẫn chưa có benchmark truy xuất POI được gán nhãn, tập đánh giá chất lượng câu trả lời, log đánh giá LLM hoặc báo cáo tải hệ thống có thể tái lập. Vì vậy, chương này trình bày lần chạy hồi quy đã hoàn thành cùng các bằng chứng khác, đồng thời đề xuất cách thu thập số đo mô hình và hệ thống còn thiếu.

Sự phân biệt quan trọng vì thường có ba loại nhận định bị nhầm lẫn. Thứ nhất, quy tắc trong mã nguồn, chẳng hạn từ chối `osm_id` trùng lặp, là hành vi đã triển khai. Thứ hai, kết quả unit test đã chạy cung cấp bằng chứng rằng một số assertion đạt trên working tree được kiểm tra. Thứ ba, chỉ số mô hình như độ chính xác câu trả lời hoặc tính khả thi của tuyến cần tập dữ liệu đánh giá xác định và kết quả đo. Lần chạy dưới đây hỗ trợ nhận định hồi quy trong phạm vi hẹp, nhưng không cung cấp loại bằng chứng thứ ba.

Quy trình đánh giá được đề xuất tách thành ba nhánh để đo riêng truy xuất, tính khả thi của lịch trình và mức độ có căn cứ của câu trả lời. Hình 5.1 tóm tắt loại bằng chứng cần thu thập; sơ đồ không mô tả thí nghiệm đã hoàn thành.



Hình 5.1: Luồng bằng chứng đánh giá đề xuất; chưa có benchmark truy xuất, chất lượng lịch trình, căn cứ câu trả lời hoặc hiệu năng

5.2. Kết quả xử lý tập dữ liệu

Bản tổng hợp đã lưu ghi nhận 110.421 bản ghi đầu vào từ ba lớp được nhập. Trong đó, 30.941 dòng vượt qua lọc ứng viên theo lớp và 30.839 định danh duy nhất được liệt kê sau khử trùng. Bản tổng hợp cung cấp số lượng theo danh mục và số bản ghi bị loại theo từng lớp, được chép lại trong Chương 3. Các giá trị này mô tả hồ sơ dữ liệu và cho thấy pipeline nhập lọc đáng kể theo tên, lớp và tọa độ.

Các số trong tệp kê khai chưa hoàn toàn tự nhất quán. Lấy 30.839 mã duy nhất khỏi 30.941 ứng viên cho ra 102 dòng trùng, trong khi trường trùng lặp tường minh báo 100. Tổng theo nhóm danh mục bằng

30.839. Đây là phát hiện về chất lượng dữ liệu, không phải kết quả đã được tự hiệu chỉnh. Lần đối soát nhập mới cần xuất từng mã OSM trùng cùng lớp nguồn xuất hiện, sau đó khớp lại số tổng trước khi báo cáo số bản ghi được chấp nhận cuối cùng.

Bản tổng hợp để null các giá trị chưa biết về giá, đánh giá và giờ mở cửa; đồng thời định nghĩa địa chỉ dự phòng từ tọa độ. Đây là lựa chọn thiết kế tích cực vì pipeline không thay thông tin thiếu bằng giá trị tự tạo. Đổi lại, mức hữu dụng của danh mục thay đổi theo trường: một điểm có thể tìm thấy theo địa lý nhưng thiếu chi tiết để trả lời địa điểm có mở cửa, giá phải chăng hoặc phù hợp để ghé thăm không. Kho mã không công bố tỷ lệ đầy đủ theo từng trường, nên chưa thể tính tỷ lệ hoàn chỉnh từ bằng chứng đã khảo sát.

5.3. Thiết kế kiểm thử chức năng trong kho

Tệp `tests/test_multi_agent_flow.py` định nghĩa chín phương thức unittest. Mô tả mô-đun cho biết kiểm thử dùng bộ mẫu và các lời gọi cơ sở dữ liệu/nhà cung cấp giả lập, không kết nối hay ghi vào PostgreSQL đã cấu hình. Bảng 5.1 tóm tắt assertion của các ca này và kết quả trong lần chạy cục bộ đã ghi nhận.

Bảng 5.1: Các ca kiểm thử hồi quy và kết quả của lần chạy cục bộ đã ghi nhận

STT	Tình huống kiểm thử	Hành vi được assertion kỳ vọng	Kết quả
1	Bộ lập lịch ngoại tuyến với tọa độ phẳng trong bộ mẫu	Bản nháp hợp lệ, thành công, có ba điểm dừng và có giá trị vĩ độ, kinh độ	ĐẠT
2	Endpoint tạo nháp với POI giả lập	HTTP 200, bản nháp hợp lệ và `is_saved=False`	ĐẠT
3	Nội dung ngày được chọn sai định dạng	HTTP 400	ĐẠT
4	Ngày chọn nằm trong quá khứ	HTTP 400 và trạng thái yêu cầu không hợp lệ	ĐẠT
5	Có vĩ độ nhưng thiếu kinh độ	HTTP 400	ĐẠT
6	Tọa độ không phải giá trị số	HTTP 400	ĐẠT
7	Giả lập lỗi cơ sở dữ liệu staging	HTTP 503 và trạng thái lỗi cơ sở dữ liệu	ĐẠT
8	Endpoint xác nhận không có Bearer token	HTTP 401	ĐẠT
9	Đăng nhập với dịch vụ xác thực giả lập	HTTP 200 cùng payload token/người dùng giả lập	ĐẠT

Các ca này tập trung vào kiểm tra đầu vào, luồng lập lịch ngoại tuyến, ánh xạ trạng thái API và một phần hành vi xác thực. Mô-đun không kiểm tra kết nối PostgreSQL trực tiếp, rollback giao dịch thật, gửi email, nhà cung cấp LLM bên ngoài, grounding Google Search, định tuyến đường thực tế hoặc tải đồng thời. Mô-đun cũng không chấm mức liên quan hay tính đúng sự thật của câu trả lời. Lần chạy đạt cung cấp bằng chứng hồi quy hữu ích nhưng không phải bằng chứng về độ tin cậy sản xuất hoặc chất lượng mô hình.

Lần chạy được thực hiện ngày 21/10/2026 từ thư mục gốc kho mã bằng Python 3.10.11 trong môi trường

ảo của dự án. Lệnh sử dụng là `venv/Scripts/python.exe -m unittest discover -s tests -p test_*.py -v`; trình chạy ghi nhận `Ran 9 tests in 0.048s; OK`. Cả chín ca trong Bảng 5.1 đều đạt, không có lỗi hay thất bại. Checkout có các thay đổi chưa commit, nên kết quả áp dụng cho working tree cục bộ tại thời điểm chạy, không đại diện cho một bản phát hành sạch có tag. Các lời gọi cơ sở dữ liệu và nhà cung cấp được giả lập; 0,048 giây là thời gian chạy bộ unit test nhỏ này, không phải benchmark độ trễ ứng dụng.

5.4. Hành vi triển khai xác nhận qua mã nguồn

Có thể xác nhận một số thuộc tính từ phần triển khai mà không cần tuyên bố kết quả benchmark:

- Endpoint lịch trình đặt `execute_api=False`; đường tạo nháp thông thường dùng bộ lập lịch xác định.
- Bộ điều phối chỉ chuyển POI truy xuất từ cơ sở dữ liệu hoặc được cung cấp tường minh cho bộ lập lịch; bộ xác thực đối chiếu định danh bản nháp với tập ứng viên.
- Hàm tuyến đường trả khoảng cách Haversine ước tính nhân 1,3 và để thời gian di chuyển ở trạng thái chưa biết.
- Bộ xác thực tách lỗi làm bản nháp không hợp lệ khỏi cảnh báo về giờ mở cửa thiếu/phức tạp và thời gian đi chưa biết.
- Endpoint bản nháp trả đối tượng chưa lưu. Một thao tác xác nhận riêng được bảo vệ bằng kiểm tra JWT của người dùng đã xác minh.
- Tìm kiếm bán kính thực hiện lọc sơ bộ hộp tọa độ, lọc Haversine chính xác, sắp theo khoảng cách rồi mới giới hạn số kết quả.

Các thuộc tính này cho thấy ranh giới an toàn phần mềm nhưng không định lượng hiệu quả thực tế. Ví dụ, kiểm tra `osm_id` của điểm dừng thuộc tập ứng viên ngăn bộ lập lịch thêm định danh ngoài tập đó. Nó không xác minh tên OSM chính xác, địa điểm còn hoạt động hay tập ứng viên đã có các lựa chọn tốt nhất.

5.5. Đề xuất đánh giá truy xuất

Nghiên cứu truy xuất tiếp theo nên bắt đầu từ tập truy vấn được gán nhãn thủ công. Mỗi câu hỏi cần một hoặc nhiều OSM ID liên quan cùng lý do liên quan; tập câu hỏi nên bao gồm tên địa điểm chính xác, loại hình, truy vấn chỉ có tên khu vực, tìm quanh tọa độ, tiếng Việt có dấu và cách diễn đạt đa ngôn ngữ. Mỗi truy vấn cần chạy trên bản sao cơ sở dữ liệu cố định với giới hạn và bán kính được ghi lại. Nhãn liên quan phải được chuẩn bị độc lập với kết quả truy xuất để tránh chấm hệ thống theo chính đầu ra của nó.

Với tập truy vấn $\mathcal{Q}$, có thể dùng các thước đo xếp hạng:

$$\text{Hit@K} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{I}\left(R_q^{(K)} \cap G_q \neq \varnothing\right), \tag{5.1}$$

$$\text{MRR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_q}, \tag{5.2}$$

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}, \tag{5.3}$$

trong đó $R_q^{(K)}$ là top-K kết quả truy xuất, G_q là tập mã liên quan và rank_q là vị trí liên quan đầu tiên. Hit@K cho biết có xuất hiện ít nhất một kết quả liên quan đã biết hay không; MRR thưởng thứ hạng cao cho kết quả liên quan đầu tiên; NDCG xử lý mức liên quan theo bậc khi người gán nhãn phân biệt kết quả chính xác, gần đúng và yếu. Bản chụp hiện tại không có giá trị nào trong các chỉ số này.

Tối thiểu, đánh giá nên so sánh tìm kiếm văn bản/danh mục SQL hiện tại với baseline chỉ dựa trên địa lý và một baseline ngữ nghĩa được mô tả rõ nếu dự định triển khai chỉ mục Chroma tùy chọn. Mọi phương pháp phải dùng cùng danh mục và bộ câu hỏi cố định. Cần báo cáo theo loại truy vấn và ngôn ngữ vì điểm tổng thể có thể che hiệu năng kém với tên địa phương, tên hành chính hoặc câu hỏi không phải tiếng Việt. Chỉ số xếp hạng không đồng nghĩa với độ chính xác thực tế của câu trả lời.

5.6. Đề xuất đánh giá lịch trình và câu trả lời

Với bản nháp lịch trình, chỉ số chính hữu ích là tỷ lệ khả thi do bên độc lập kiểm tra:

$$\text{FeasibilityRate} = \frac{\text{số bản nháp đạt mọi ràng buộc cứng đã định nghĩa}}{\text{số bản nháp được rà soát}}. \tag{5.4}$$

Danh sách ràng buộc cứng cần gồm sự hiện diện trong tập ứng viên, điểm dừng trùng lặp, tọa độ hợp lệ, khoảng giờ hợp lệ, chồng lấn, tổng thời gian trong ngày và các quy tắc giờ mở cửa có thể kiểm tra. Do bộ xác thực chỉ triển khai một phần ràng buộc du lịch thực tế, người đánh giá hoặc bộ kiểm tra độc lập cũng cần xem xét tính hợp lý của tuyến, khả năng tiếp cận, lịch mở cửa và quỹ thời gian. Kết quả `valid=True` của chính bộ xác thực không thể là chuẩn sự thật duy nhất để đo bộ xác thực.

Chất lượng câu trả lời có căn cứ nên được đánh giá ở mức từng mệnh đề. Người rà soát gắn nhãn mỗi phát biểu thực tế là được bản ghi OSM hỗ trợ, có nguồn web trích dẫn hỗ trợ, không có căn cứ hoặc bị mâu thuẫn. Cách này phân biệt được tên địa điểm đúng với giá vé bịa đặt hay phát biểu giờ mở cửa thiếu điều kiện. Có thể bổ sung đánh giá của con người về mức liên quan, rõ ràng và chất lượng ngôn ngữ. BLEU và ROUGE là các thước đo độ trùng lặp văn bản đã được nghiên cứu [19], [20], nhưng riêng chúng không đủ cho tư vấn du lịch dạng mở: một bản diễn đạt trung thành có thể trùng từ thấp, trong khi câu trả lời trôi chảy có thể lặp từ của câu chuẩn mà vẫn chứa phát biểu sai.

Đánh giá tuyến đường cần có nguồn tuyến tham chiếu và phương tiện di chuyển được khai báo. Với mỗi cặp điểm dừng, so sánh khoảng cách ước tính của hệ thống với khoảng cách mạng đường và báo cáo phân phối sai số. Vì hàm tuyến đường hiện không trả thời lượng ước tính, chưa thể tính sai số thời gian cho luồng lịch trình mặc định. Nếu tích hợp API định tuyến, cần ghi ngày truy vấn, phương tiện, tùy chọn tránh đường và phiên bản API vì các yếu tố đó ảnh hưởng đến tuyến và giả định giao thông.

5.7. Đề xuất đánh giá hiệu năng và độ tin cậy

Bản chụp kho không có số đo độ trễ, thông lượng, lỗi nhà cung cấp hoặc kiểm thử tải. Nghiên cứu vận hành có thể lặp lại cần nêu CPU/RAM máy chủ, phiên bản Python, khóa phụ thuộc, phiên bản PostgreSQL và số dòng, điều kiện mạng, mô hình/nhà cung cấp cùng khu vực, lịch đồng thời, thời lượng khởi động và tổng số yêu cầu. Cần tách tìm kiếm chỉ dùng cơ sở dữ liệu, tạo nháp xác định, sinh qua nhà cung cấp trực tiếp và giao dịch xác nhận vì các tuyến này có chi phí, độ trễ khác nhau, không nên gộp thành một con số.

Với từng endpoint và mức đồng thời, nên báo cáo độ trễ trung vị và phân vị 95, số yêu cầu hoàn thành mỗi giây, số lượng mã trạng thái, tỷ lệ timeout, mức bão hòa connection pool nếu có cấu hình và lỗi nhà cung cấp ngoài. Mỗi phép đo cần lặp lại và thể hiện độ biến động. Kiểm thử tải với mô hình giả không chứng minh hành vi của nhà cung cấp thật; máy chủ phát triển cục bộ bật Flask debug cũng không đại diện cho triển khai WSGI sản xuất.

5.8. Thảo luận kết quả hiện tại

Bằng chứng hiện có hỗ trợ một kết luận thận trọng. Bản tổng hợp tiền xử lý mô tả tập ứng viên đã lọc có quy mô đáng kể; mô hình SQL và công cụ truy vấn cung cấp cơ chế tìm kiếm địa lý cụ thể; bộ lập lịch có đường chạy xác định ngoại tuyến; bộ xác thực mã hóa một số biện pháp bảo vệ cấu trúc và thời gian hữu ích. Bộ kiểm thử hồi quy an toàn, dùng giả lập có mặt cho các tình huống yêu cầu/phản hồi tiêu biểu. Đây là bằng chứng triển khai có ý nghĩa cho nguyên mẫu đồ án.

Bằng chứng không hỗ trợ các số liệu từng xuất hiện trong bản nháp trước về hit rate truy xuất, BLEU/ROUGE đa ngôn ngữ, phần trăm khả thi lịch trình, không có hallucination hoặc thông lượng đồng thời. Kho không có tập kiểm thử gán nhãn hoặc log chạy tương ứng. Các giá trị đó đã được loại bỏ thay vì giữ lại như kết quả đo. Tương tự, việc `chromadb`, `sentence-transformers` và `pgvector` xuất hiện trong phụ thuộc hoặc image container không chứng minh có benchmark truy xuất vector đang hoạt động ở luồng lập lịch mặc định.

Một số giới hạn triển khai ảnh hưởng trực tiếp đến cách diễn giải:

1. **Cá nhân hóa bộ lập lịch còn hạn chế.** Baseline dùng lịch khung giờ cố định và phân loại ăn uống thô. Trường sở thích và ngân sách được nhận nhưng chưa được chấm điểm hoặc xác thực toàn diện.
2. **Thử lại chưa chắc cải thiện bản nháp.** Bộ điều phối có thể trả phản hồi xác thực tối đa hai lần thử lại. Cách chọn xác định không tìm kiếm tường minh phương án khác để xử lý từng lỗi; giá trị thực tế của cơ chế thử lại cần được kiểm thử.
3. **Chưa biết tính khả thi của đường đi.** Khoảng cách gần đúng được trả mà không có thời gian lái xe. Khoảng trống giữa các lượt ghé không thể được chứng nhận đủ cho việc di chuyển ở luồng mặc định.
4. **Độ mới và chi tiết dữ liệu không đồng đều.** Giờ và giá null hạn chế tư vấn cụ thể; số lượng bản ghi không chứng minh rằng dữ liệu mới hoặc phù hợp du lịch.
5. **Sai khác số liệu tệp kê khai chưa được xử lý.** Cần kiểm toán nhập ở cấp bản phát hành để đối soát số trùng và xác minh số dòng staging được triển khai.

5.9. Trình tự đánh giá khuyến nghị

Chu kỳ đánh giá tiếp theo nên đi theo thứ tự nhằm hạn chế yếu tố gây nhiễu:

1. Chạy lại bộ nhập ở chế độ dry-run từ bản trích xuất OSM đã lưu; xuất số lượng từng lớp, tệp mã trùng tường minh và tổng sau đối soát.
2. Nạp bản sao cố định vào PostgreSQL thử nghiệm, ghi phiên bản lược đồ, truy vấn tổng số dòng và độ đầy đủ trường.
3. Lưu kết quả unittest đã ghi nhận cùng trạng thái checkout chính xác. Sau đó bổ sung kiểm thử trực tiếp ranh giới bộ xác thực và giao dịch của dịch vụ xác nhận, đồng thời chạy lại sau mỗi thay đổi mã liên quan.
4. Tạo bộ truy vấn gán nhãn thủ công, làm mù, rồi chấm riêng truy xuất văn bản, danh mục và bán kính.
5. Đánh giá bản nháp xác định trên tập yêu cầu cố định. So sánh đầu ra bộ xác thực với kiểm toán độc lập bằng người hoặc quy tắc tham chiếu, báo cáo riêng từng ràng buộc.
6. Chỉ đánh giá chatbot và bộ lập lịch dùng nhà cung cấp sau khi ghi tên mô hình, prompt, tham số, khu vực nhà cung cấp và đầu ra yêu cầu/phản hồi theo quy trình bảo vệ riêng tư.
7. Kiểm thử hiệu năng và lỗi trên tiến trình WSGI cùng cơ sở dữ liệu gần với sản xuất; công bố cấu hình cùng mọi kết quả độ trễ và lỗi.

5.10. Tóm tắt chương

Báo cáo ghi nhận thống kê xử lý dữ liệu mô tả, hành vi có thể truy xuất trong mã nguồn và một lần chạy thành công bộ hồi quy gồm chín ca (9 đạt, 0 trượt, 0 lỗi). Vì cơ sở dữ liệu và nhà cung cấp được giả lập, lần chạy không xác nhận tích hợp trực tiếp, chất lượng mô hình hay độ tin cậy sản xuất. Kết luận hệ thống vững chắc nhất vẫn là đã triển khai luồng truy xuất từ cơ sở dữ liệu, tạo nháp ngoại tuyến và xác thực, đồng thời nêu rõ bất định về giờ mở cửa thiếu và thời gian đi đường. Phần thực nghiệm còn cần xác minh dữ liệu nhập, đo mức liên quan truy xuất, kiểm toán độc lập tính khả thi lịch trình và benchmark riêng từng luồng vận hành.

Tài liệu tham khảo

[1] M. Haklay and P. Weber, "Openstreetmap: User-generated street maps," *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008.

[2] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008. [Online]. Available: https://arxiv.org/abs/1706.03762.

[3] T. B. Brown, B. Mann, N. Ryder, *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901. [Online]. Available: https://arxiv.org/abs/2005.14165.

[4] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401.

[5] Y. Gao, Y. Xiong, X. Gao, *et al.*, *Retrieval-augmented generation for large language models: A survey*, arXiv:2312.10997, 2023. [Online]. Available: https://arxiv.org/abs/2312.10997.

[6] J. Borràs, A. Moreno, and A. Valls, "Intelligent tourism recommender systems: A survey," *Expert Systems with Applications*, vol. 41, no. 16, pp. 7370–7389, 2014.

[7] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou, "Mobile recommender systems in tourism," *Journal of Network and Computer Applications*, vol. 39, pp. 319–333, 2014.

[8] K. H. Lim, J. Chan, C. Leckie, and S. Karunasekera, "Personalized tour recommendation based on user interests and points of interest visit durations," in *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-15)*, 2015, pp. 1778–1784. [Online]. Available: https://www.ijcai.org/Abstract/15/253.

[9] V. Karpukhin, B. Oguz, S. Min, *et al.*, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781. [Online]. Available: https://aclanthology.org/2020.emnlp-main.550/.

[10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992. [Online]. Available: https://aclanthology.org/D19-1410/.

[11] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[12] Q. Wu, G. Bansal, J. Zhang, *et al.*, *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*, arXiv:2308.08155, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155.

[13] S. Hong, M. Zhuge, J. Chen, *et al.*, *MetaGPT: Meta programming for a multi-agent collaborative framework*, arXiv:2308.00352, 2023. [Online]. Available: https://arxiv.org/abs/2308.00352.

[14] S. Yao, J. Zhao, D. Yu, *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629.

[15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 8634–8652. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

[16] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 158, 1984.

[17] D. L. Applegate, R. E. Bixby, V. Chvatal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

[18] P. Vansteenwegen, W. Souffriau, and D. V. Oudheusden, "The orienteering problem: A survey," *European Journal of Operational Research*, vol. 209, no. 1, pp. 1–10, 2011.

[19] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002, pp. 311–318. [Online]. Available: https://aclanthology.org/P02-1040/.

[20] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/.

A. Từ điển dữ liệu và ghi chú tái lập

A.1. Các trường của bản ghi staging

Mô hình staging OSM lưu bản ghi theo nguồn, không giả định rằng mọi đối tượng OSM có mức chi tiết như điểm tham quan đã biên tập. Bảng A.1 mô tả các cột quan trọng trong `OsmAttractionStaging` và migration staging.

Bảng A.1: Từ điển trường của bảng staging OSM

Trường	Kiểu / ràng buộc	Ý nghĩa
`id`	Khóa chính số nguyên	Định danh nội bộ của dòng staging.
`osm_id`	Chuỗi, duy nhất, không null	Định danh đối tượng nguồn ổn định để khử trùng và truy vết.
`osm_source_layer`	Chuỗi, không null	Tên lớp shapefile nguồn.
`osm_code`	Số nguyên, tùy chọn	Mã lớp nguồn dạng số nếu có.
`fclass`	Chuỗi, không null	Lớp đối tượng OSM gốc.
`name`	Chuỗi, không null	Tên hiển thị giữ từ bản ghi nguồn.
`category`	Chuỗi, không null	Danh mục du lịch đã chuẩn hóa.
`latitude`, `longitude`	Double precision, không null	Tọa độ địa lý WGS 84 theo độ thập phân.
`address`	Chuỗi, không null	Địa chỉ nguồn nếu có; nếu không thì dùng chuỗi dự phòng nêu rõ tọa độ.
`description`	Văn bản, tùy chọn	Mô tả ngắn suy ra từ thông tin OSM được giữ lại.
`price`, `rating`	Số, tùy chọn	Giá trị chưa biết được giữ null theo chuẩn nhập.
`opening_hours`	Chuỗi, tùy chọn	Chuỗi giờ OSM; định dạng có thể phức tạp hoặc bị thiếu.
`phone_number`, `image_url`	Chuỗi, tùy chọn	Siêu dữ liệu liên hệ và hình ảnh khi có.
`language`	Chuỗi	Nhãn ngôn ngữ được quy trình nhập gán.
`tags`, `aliases`	JSONB	Thẻ tìm kiếm và tên thay thế.
`raw_properties`	JSONB	Giá trị thuộc tính gốc để truy vết.
`imported_to_attraction`, `attraction_id`	Boolean và chuỗi	Trạng thái chuyển đổi và liên kết tùy chọn tới attraction lõi của ứng dụng.

Trường	Kiểu / ràng buộc	Ý nghĩa
`created_at`, `updated_at`	Timestamp	Thời điểm tạo và cập nhật dòng.

A.2. Danh sách kiểm tra tái lập nhập dữ liệu

Tập lệnh nhập được thiết kế để hỗ trợ rà soát dry-run trước khi bật ghi dữ liệu. Hồ sơ phát hành nên bao gồm:

1. thời điểm trích xuất và thông báo giấy phép từ gói dữ liệu nguồn;
2. checksum tệp cho ba lớp đầu vào;
3. phiên bản Python chính xác cùng phiên bản phụ thuộc đã cài;
4. tùy chọn bộ nhập, gồm chế độ khử trùng và kích thước lô;
5. số dòng thô, số bị loại theo từng bộ lọc và số giữ lại theo lớp;
6. báo cáo trùng lặp đọc được bằng máy, liệt kê từng OSM ID bị lặp;
7. tổng dòng staging cuối cùng và mẫu bản ghi truy vấn từ PostgreSQL;
8. phiên bản lược đồ cơ sở dữ liệu hoặc migration dùng khi nạp.

Dữ liệu này sẽ giải quyết điểm chưa rõ giữa số ứng viên, số duy nhất và số trùng trong bản tổng hợp hiện tại. Bản phát hành an toàn cũng nên giữ báo cáo dry-run bên cạnh báo cáo lần áp dụng để so sánh thay đổi dự kiến với dữ liệu đã ghi thật.

B. Bề mặt API và ranh giới yêu cầu

B.1. Các tuyến du lịch và lịch trình chính

Bảng B.1 tóm tắt các tuyến trọng tâm của đề tài. Đường dẫn và mục đích lấy từ namespace Flask-RESTX cùng controller đã đăng ký. Trang Swagger tương tác tại `/docs` là tham chiếu chính khi cần xem lược đồ và trạng thái phản hồi chi tiết.

Bảng B.1: Các endpoint tìm kiếm du lịch và lập lịch chính

Phương thức	Đường dẫn	Mục đích / ranh giới
POST	`/api/travel-chatbot/search`	Nhận câu hỏi du lịch bằng ngôn ngữ tự nhiên và cờ tìm kiếm web tùy chọn; trả kết quả tìm kiếm cùng câu trả lời được sinh.
GET	`/api/travel-chatbot/metadata`	Đọc siêu dữ liệu địa điểm có phân trang qua controller chatbot du lịch.
GET	`/api/travel-chatbot/embeddings`	Tuyến embedding cũ còn trong controller; không phải cơ chế truy xuất mặc định của bộ lập lịch.
POST	`/api/itinerary/plan-draft`	Tạo bản nháp; cần ngày chọn và câu truy vấn văn bản hoặc cặp tọa độ hợp lệ; không lưu bản nháp.
POST	`/api/itinerary/multi-agent/plan-draft`	Bí danh cho thao tác lập bản nháp.
POST	`/api/itinerary/confirm`	Xác nhận và lưu bản nháp bằng tài khoản đã xác minh từ Bearer JWT.
POST	`/api/itinerary/multi-agent/confirm`	Bí danh cho thao tác xác nhận.
GET / POST / DELETE	`/api/itinerary/...`	Các thao tác hiện có để liệt kê, xem chi tiết, tạo, xóa và cập nhật mục lịch trình đã lưu. Xem Swagger để biết phương thức và payload từng tài nguyên.

B.2. Xác thực yêu cầu tạo bản nháp

Controller tạo nháp nhận ngày, truy vấn văn bản tùy chọn, nhãn điểm đến/khu vực, danh mục, tọa độ, bán kính, sở thích, ngân sách, số điểm dừng tối đa và prompt tùy chọn. Controller từ chối cặp tọa độ chỉ có một giá trị, tọa độ không phải số hoặc ngoài phạm vi, bán kính ngoài khoảng hỗ trợ và nhãn khu vực không có tâm tọa độ. Quy tắc cuối quan trọng vì controller lịch trình không chuyển tên khu vực thành tọa độ.

Bộ điều phối còn kiểm tra giới hạn yêu cầu, gồm số điểm dừng từ một tới mười và giới hạn kết quả tìm kiếm từ một tới một trăm. Ngày quá khứ bị từ chối trong tầng điều phối. Lỗi tìm kiếm được ánh xạ sang trạng thái phản hồi tường minh; cơ sở dữ liệu không khả dụng được báo riêng với đầu vào không hợp lệ và kết quả tìm kiếm rỗng nhưng hợp lệ.

B.3. Ranh giới xác nhận bản nháp

Tuyến xác nhận yêu cầu mã bản nháp đúng định dạng controller mong đợi, ngày chọn và danh sách mục không rỗng. Tuyến đọc Bearer token trong header Authorization, xác minh chữ ký cùng hạn dùng, lấy người dùng đã xác thực rồi kiểm tra email đã xác minh. Mã người dùng có trong JSON không được chấp nhận làm căn cứ sở hữu.

Dịch vụ xác nhận kiểm tra lại chi tiết bản nháp theo bản ghi staging và lưu lịch trình quan hệ. Lần xác nhận mới thành công trả HTTP 201; yêu cầu trùng đã nhận diện có thể trả HTTP 200. Lỗi xác thực dữ liệu, thiếu nội dung, lỗi phân quyền và lỗi giao dịch được biểu diễn bằng các nhóm phản hồi riêng. Mã trạng thái ở cấp controller là một phần hợp đồng API và cần ổn định nếu thay đổi cách triển khai bộ lập lịch.

C. Danh mục kiểm thử hồi quy và đánh giá

C.1. Các ca hồi quy hiện có

Chín phương thức kiểm thử trong `tests/test_multi_agent_flow.py` tạo thành lớp hồi quy ngoại tuyến nhỏ. Bộ mẫu có 15 bản ghi theo cấu trúc OSM, cho phép kiểm tra hành vi bộ lập lịch mà không cần cơ sở dữ liệu sản xuất. Các hàm nhà cung cấp và cơ sở dữ liệu được giả lập khi cần cô lập controller. Bộ kiểm thử đã được chạy từ thư mục gốc kho mã bằng môi trường ảo của dự án; kết quả được ghi dưới đây. Lần chạy không dùng hoặc ghi vào cơ sở dữ liệu trực tiếp của người dùng.

Bảng C.1: Ánh xạ kiểm thử với yêu cầu và kết quả đã ghi nhận

Mã	Nhóm yêu cầu	Assertion có trong mã nguồn	Kết quả
T-01	Lập lịch ngoại tuyến	Bộ lập lịch trả bản nháp hợp lệ, thành công, có ba điểm và có giá trị vĩ độ/kinh độ.	ĐẠT
T-02	Hợp đồng API tạo nháp	Tìm kiếm staging giả lập trả HTTP 200 và phản hồi được đánh dấu chưa lưu.	ĐẠT
T-03	Xác thực đầu vào	Chuỗi ngày không hợp lệ bị từ chối bằng HTTP 400.	ĐẠT
T-04	Xác thực đầu vào	Ngày đã qua bị từ chối bằng HTTP 400.	ĐẠT
T-05	Xác thực tọa độ	Có vĩ độ nhưng thiếu kinh độ bị từ chối bằng HTTP 400.	ĐẠT
T-06	Xác thực tọa độ	Giá trị tọa độ không phải số bị từ chối bằng HTTP 400.	ĐẠT
T-07	Ánh xạ lỗi	Lỗi cơ sở dữ liệu staging giả lập được ánh xạ sang HTTP 503.	ĐẠT
T-08	Phân quyền xác nhận	Yêu cầu xác nhận không có Bearer token được ánh xạ sang HTTP 401.	ĐẠT
T-09	Controller đăng nhập	Phản hồi xác thực giả lập được trả HTTP 200 kèm token và trường người dùng.	ĐẠT

Lần chạy được thực hiện ngày 21/10/2026 bằng Python 3.10.11 trong môi trường ảo của dự án. Lệnh chạy là `venv/Scripts/python.exe -m unittest discover -s tests -p test_*.py -v`; trình chạy ghi nhận `Ran 9 tests in 0.048s; OK`. Cả T-01 đến T-09 đều đạt, không có lỗi hay thất bại. Working tree còn các thay đổi chưa commit, vì vậy kết quả chỉ áp dụng cho bản cục bộ tại thời điểm chạy, không đảm bảo cho revision về sau. Cơ sở dữ liệu và nhà cung cấp được giả lập. Các lần chạy tiếp theo nên ghi trạng thái checkout và đầu ra cụ thể; không đưa token, mật khẩu, hồ sơ người dùng hoặc payload nhạy cảm vào log.

C.2. Kiểm thử bổ sung nên có trước phát hành

Kiểm thử hiện tại tập trung vào tạo nháp thành công và một số ranh giới yêu cầu. Bộ hồi quy rộng hơn nên bao gồm:

- **Độ đúng của tìm kiếm:** so sánh kết quả bán kính SQL với phép tính Haversine độc lập; xác minh thứ tự gần trước và việc chỉ giới hạn sau khi lọc bán kính.
- **Nhập dữ liệu:** kiểm tra ID trùng giữa và trong các lớp, tọa độ lỗi, tên trống, ánh xạ danh mục, giữ null và tính idempotent của upsert trên cơ sở dữ liệu bỏ đi được.
- **Ranh giới bộ xác thực:** kiểm tra nửa đêm, giờ sai định dạng, lượt ghé không thời lượng, chồng lấn tại điểm bằng nhau, giới hạn 12 giờ, khung giờ đơn giản/phức tạp và tọa độ thiếu.
- **Ngữ nghĩa bộ lập lịch:** xác minh tính duy nhất, giới hạn số điểm dừng, độ bao phủ ứng viên, kết quả xác định trên bộ mẫu và hành vi khi thiếu nhóm ăn uống hoặc tham quan.
- **Thử lại:** tạo bộ ứng viên gây lỗi nghiêm trọng rồi kiểm tra phản hồi có làm thay đổi lần lập kế hoạch sau hay chỉ lặp lại bản cũ.
- **Lưu trữ:** xác minh xác nhận kiểm tra lại dữ liệu staging, ánh xạ OSM ID tới attraction lõi, gán đúng người dùng đã xác thực và rollback toàn bộ khi ép lỗi.
- **Tích hợp nhà cung cấp:** dùng giả lập để kiểm tra JSON cấu trúc lỗi, phản hồi hạn mức, timeout, dự phòng, thiếu thông tin xác thực và an toàn log.
- **Kiểm soát truy cập:** kiểm tra JWT hết hạn, sai định dạng hoặc bị sửa, người dùng chưa xác minh, xác nhận lịch của người khác, OTP hết hạn và giới hạn tốc độ.

C.3. Mẫu ghi nhận đánh giá

Khi thu thập phép đo, bảng kết quả cần đủ ngữ cảnh để người đọc diễn giải độc lập. Bảng C.2 là mẫu ghi nhận để dùng trong lần chạy tương lai, không phải báo cáo kết quả đã hoàn thành.

C.4. Lệnh phát triển cục bộ

README dự án mô tả trình tự phát triển tổng quát dưới đây. Chạy lệnh từ thư mục gốc kho mã sau khi đã cấu hình biến môi trường cục bộ:

```
python -m venv venv
pip install -r requirements.txt
docker compose -f docker-compose.dev.yml up -d
python scripts/import_all_osm_poi.py --apply --batch-size 1000
python main.py
```

Listing C.1: Trình tự phát triển cục bộ được dự án mô tả

Khi rà soát bản trích xuất mới, trước tiên nên chạy bộ nhập mà không có `--apply`. Cần kiểm tra và áp dụng migration của kho trước khi nhập. Không để máy chủ phát triển bật debug công khai. Với thí nghiệm có thể tái lập, hãy lưu riêng đầu ra lệnh và siêu dữ liệu cấu hình, không lưu bí mật.

Bảng C.2: Mẫu ghi nhận cho một lần đánh giá trong tương lai

Trường	Giá trị cần ghi
Revision nguồn	Git commit hoặc định danh bản lưu trữ
Phiên bản dữ liệu	Thời điểm trích OSM, checksum tệp, số dòng cơ sở dữ liệu cuối
Môi trường	Hệ điều hành, CPU, bộ nhớ, phiên bản Python và PostgreSQL
Cấu hình nhà cung cấp	Model ID, vùng, nhiệt độ, giới hạn đầu ra, chế độ grounding, phiên bản prompt
Tập truy vấn	Số truy vấn, ngôn ngữ, nhóm danh mục, quy trình gán nhãn
Chỉ số tìm kiếm	Hit@K, MRR, NDCG theo ngôn ngữ, danh mục và kiểu truy vấn địa điểm
Chỉ số lịch trình	Tỷ lệ khả thi kiểm tra độc lập và vi phạm từng ràng buộc
Đánh giá câu trả lời	Phát biểu có căn cứ/thiếu căn cứ, điểm liên quan và độ rõ ràng
Chỉ số vận hành	Độ trễ p50/p95, thông lượng, lỗi và timeout theo endpoint
Số lần lặp	Số lần chạy độc lập và cách tính độ biến động hoặc khoảng tin cậy
Hạn chế	Nhãn thiếu, nhà cung cấp lỗi, lỗ hổng dữ liệu, sai lệch quy trình

C.5. Ghi chú kết thúc

Danh mục này giúp người đọc kiểm tra ranh giới bằng chứng của báo cáo. Báo cáo hiện ghi nhận thống kê tập dữ liệu và một lần chạy hồi quy cục bộ gồm chín ca đều đạt. Cần bổ sung các lần đánh giá có gán nhãn, tích hợp trực tiếp và hiệu năng cùng phiên bản và phương pháp; sau đó mới thay chỉ số dự kiến bằng số đo thực. Khi chưa có các ghi nhận ấy, không đưa số liệu chất lượng mô hình hoặc thông lượng chưa được đo vào báo cáo cuối.

6. Nhận xét và đánh giá của giảng viên

Bảng 6.1: Phiếu đánh giá đồ án và nhận xét của giảng viên

HẠNG MỤC	ĐIỂM	NHẬN XÉT
Ý TƯỞNG	____/10	
ỨNG DỤNG	____/30	
KẾT QUẢ	____/30	
QUẢN LÝ DỰ ÁN	____/10	
THUYẾT TRÌNH VÀ BÁO CÁO	____/20	
TỔNG ĐIỂM	____/100	

Chữ ký giảng viên

Ngày

TS. Trần Quang Quý